ASD Group 18 Technical Report

https://github.com/rigels51/ASD-Project-Group18

1.0 Project Overview

1.1 Project Overview and Objectives

The project is a University Management System developed as a collection of independent microservices. Each group member was responsible for developing an individual university management feature while following a shared architecture and frontend design.

The main objective of the project was to develop a functional integrated application that supports common university management tasks such as managing student records, staff, timetables, courses and enrolments. Each feature was designed with separate frontend, backend/API and database responsibilities to maintain modularity and make the system easier to integrate.

Another major objective was to incorporate local AI functionality into the individual features. The AI Mode allows users to ask natural-language questions about application data, with current database information provided as context to reduce unsupported responses.

Beyond AI Mode, each feature also provides an MCP Mode for structured, tool-based queries against live data and a RAG Mode for citation-backed, retrieval-grounded answers, both powered by shared MCP and RAG servers

The project also aimed to demonstrate modern software development practices, including Agile development, GitHub-based collaboration, Docker containerisation, REST APIs, automated GitHub Actions workflows and the Plan → Act → Observe → Adapt agentic workflow.

1.2 Team Members

Student 1: Rigel 25876487

Student 2: Jeriko Arceo 24806008

Student 3: Lazizbek Ismoilov 14567426

Student 4: Yi Zhang 25402140

Student 5: Vu Tien Thanh Nguyen 14673154

1.3 Individual Feature Allocation (i)

Student 1: Rigel Rivamonte

This feature manages university student records. Users can view, search, create, update, and delete student information. Each record includes a student ID, name, course, year level, email, phone number, GPA, and academic status.

Within Release 1, the Student Records Management feature has been implemented with an MCP Mode, and RAG Mode. This allows the feature to now answer natural-language questions using live student data, exposes read-only MCP tools (student count, list, lookup by ID, filter by status), and provides citation-backed RAG answers grounded in retrieved student records.

Student 2: Jeriko Arceo

Feature: Staff Management

This feature is responsible for managing staff records for the University management application. The feature allows the user to view, create, edit, and delete staff member details from the university. It also allows staff to be assigned to classes.

Its main functions include:

- Create staff
- Read staff
- Update staff
- Delete staff
- AI agent to assist with assigning staff
- Get staff data
- Get subject data
- Assign staff to classes

Student 3: Lazizbek Ismoilov

Feature: Timetable and Classes Management

This feature manages when and where each course/subject session runs. Students and lecturers can browse and filter the timetable; admin users can create, update, and delete sessions; the system automatically detects room/time scheduling clashes; and a local AI agent answers natural-language scheduling questions grounded in the live timetable data.

- Its main functions include:

- Create a new class session (course code, day, time, room, semester)
- Read / browse all sessions and filter by course code
- Update an existing session's room or time
- Delete a session
- Automatically detect scheduling clashes (same day, room, overlapping time)
- AI agent to answer natural-language scheduling questions, grounded in real timetable data
- Role-based views for Student, Lecturer, and Admin
- Get sessions by course code (cross-feature lookup point for Course Catalogue)

For Release 1, the Timetable feature was extended with MCP Mode and RAG Mode. MCP Mode lets the frontend call three read-only Timetable tools on the shared local MCP server (check_room_availability, get_sessions_by_course and find_timetable_clashes) through the Timetable backend/API and display the structured result. RAG Mode answers scheduling-policy questions from the shared RAG server's "timetable" knowledge domain, with source citations, a confidence category, and an insufficient-context response when no relevant policy is found. I also built the shared agentic loop's MCP and RAG validation modes and the RAG server's multi-domain routing.

Student4: Yi Zhang

Feature: Course and Enrollment Management

My responsibility was to develop the Course and Enrollment Management feature. It allows users to manage courses and student enrolments, including viewing, creating, updating, and deleting records.

The feature also includes a local AI Mode that can answer questions using current course and enrolment data.

It was implemented using frontend, Flask backend, SQLite database, Docker, Ollama, and GitHub Actions.

For Release 1, the Course and Enrollment Management feature was extended with MCP Mode and RAG Mode. MCP Mode connects the frontend and backend to the shared local MCP server for structured course and enrolment queries. RAG Mode connects to the shared RAG server to retrieve relevant course information and generate grounded answers with citations and confidence information.

Student 5: Vu Tien Thanh Nguyen

Feature: Assessment & Grades Management

My responsibility was to design and implement the Assessment & Grades Management feature of the University Management System. This feature manages course assessments and student results. It allows users to view, search, create, update, and delete assessment and grade records.

The feature also includes an AI Mode that allows users to ask natural-language questions about assessments, grades, and student performance. The AI retrieves current assessment and grade information before generating a response using the Plan → Act → Observe → Adapt workflow.

The main functions include:

- View and search assessments by course, assessment type, or keyword
- View individual assessment details
- Create new assessments
- Update existing assessments
- Delete assessments
- View student grade records
- Record new student grades
- Update and delete grade records
- Search grades by student ID or course
- Validate student IDs using the Student Records Management service
- Retrieve course information from the Course Management service
- Use AI Mode to answer questions about assessments, grades, and student performance
- Use current database information to provide grounded AI responses

The feature was implemented as three separate microservices: an Nginx frontend service, Flask backend/API service, and Flask/SQLite database service. Docker is used to containerise each service, while Ollama with Qwen is used for the local AI functionality.

For Release 1, the Assessment & Grades Management feature was extended with MCP Mode and RAG Mode using the shared project-level MCP and RAG servers. MCP Mode provides read-only access to assessment and grade information through three predefined tools: list_course_assessments, get_student_grade_summary, and get_upcoming_assessments. RAG Mode provides retrieval-grounded answers using assessment records, anonymised course-level grade statistics, and the assessment policy document. RAG responses include citations, source

information, and confidence levels, and the system returns insufficient evidence when the retrieved information does not support the question.

2.0 Project Analysis and Planning

2.1 Agile Methodology and sprint structure

Sprints	What will be done	Expected Outcome
Sprint 1	Team formation, topic approval, repository setup, Docker setup, and feature planning	Shared repository and development environment ready
Sprint 2	Database design, frontend development, backend API development, and CRUD implementation	Individual features able to create, read, update, and delete records
Sprint 3	AI Mode integration, applying same CSS design to all pages, Docker containerisation, testing, and shared Docker Compose integration	Release 0 integrated application completed
Sprint 4	Develop MCP and RAG components, connect services to shared retrieval tools, and improve database-grounded AI responses.	MCP and RAG integration is available for Release 1.
Sprint 5	Implement multi-agent workflow, complete testing, collect evidence, resolve outstanding integration issues, and prepare cloud deployment.	Release 2 application, testing evidence, and deployment are completed.

2.2 Sprint Backlog - Functional and Non-Functional Requirements (i)

Rigel Rivamonte

Functional Requirements	Non-functional Requirements
• The system shall provide MCP Mode for users to query student data through predefined tools, including student count, listing students, retrieving a student by ID, and listing students by status. • The system shall provide RAG Mode for users to ask natural-language questions about student records and receive answers grounded in retrieved student data. • The system shall allow users to retrieve relevant student record context for a query without generating a full AI answer. • The system shall allow the RAG index to be refreshed so that retrieval reflects the current student records. • The system shall return a fixed "insufficient context" response in RAG Mode when retrieval confidence is low, rather than generating an unsupported answer.	• **Interoperability:** MCP tools shall expose a restricted, well-defined set of student data operations so other services can safely query student data through the shared MCP server. • **Reliability:** MCP and RAG endpoints shall return 503 when the respective service is disabled or unreachable. • **Testability:** The feature shall support automated unit testing for core student-record functions. GitHub Actions shall validate Python syntax and build Docker images through the RigelRivamonte.yml workflow. • Security: MCP tools shall be read-only; no create, update, or delete operation shall be exposed through MCP Mode. • RAG Grounding: RAG Mode answers shall cite the source chunk ID(s) used, and shall abstain with a fixed message when retrieval confidence is low. • MCP Tool Boundaries: The backend shall only expose an explicit allowlist of MCP tools to the UI; calls to any tool outside the allowlist shall be rejected. • MCP and RAG integrations shall be independently toggled via the MCP_ENABLED and RAG_ENABLED environment variables without requiring code changes.

Jeriko Arceo

Functional Requirements	Non-functional Requirements
• The system shall allow users to create a new staff record (e.g. staff ID, name, email, department, employment type) • The system shall allow users to view/read a list of all staff members and view individual staff details. • The system shall allow users to update existing staff records • The system shall allow users to delete a staff record from the system. • The system shall validate staff input data (e.g. required fields, valid email format, unique staff ID) before saving to the database. • The system shall store and retrieve staff data from a dedicated Staff database • The frontend shall communicate with the backend/API via REST endpoints (GET, POST, PUT/PATCH, DELETE) to perform CRUD operations. • The system shall integrate with the AI-Mode to allow AI-assisted interaction — e.g. a natural-language query such as "list all staff in the Computer Science department" or "summarise staff workload by department." • The system shall display staff data in the unified frontend UI, following the shared CSS theme and accessible from the unified home page • The system shall provide search/filter functionality for staff • The system shall demonstrate the Plan → Act → Observe → Adapt agentic	• Performance – The staff list page shall load within 2–3 seconds under normal local Docker execution. • Usability – The UI shall be simple, consistent with the shared team CSS theme, and usable without training. • Reliability – CRUD operations shall complete successfully with appropriate error handling and user feedback (e.g. failed save, duplicate ID). • Maintainability – Code shall follow modular structure (separate frontend, backend, database layers) to support incremental extension in Release 1/2. • Portability – The microservice shall be fully containerised with Docker and run successfully via docker-compose up alongside other services. • Security – Input shall be sanitised to prevent SQL injection/XSS; sensitive staff data (e.g. contact info) shall not be exposed in logs. • Scalability – The database design shall support growth beyond the initial 10 records without requiring schema changes. • Availability – The microservice shall be available whenever the Docker Compose stack is running, with no manual intervention needed. • Interoperability – The backend/API shall expose REST endpoints in a consistent format so other student microservices (or future MCP/RAG

workflow when processing an AI-assisted staff-related request. • The system shall allow users to update/reassign a staff member's course allocation (e.g. change or add a course assignment). • AI responses shall be grounded using current staff records retrieved from the database before the answer is generated. • The system shall expose staff data access as a callable MCP tool the LLM can invoke on demand. • The AI shall base factual claims about staff only on retrieved records and shall not invent staff members that aren't present in them.	integration) can consume staff data if needed. • Testability – The microservice shall support GitHub Actions CI validation (build/test) via student-2.yml. • Performance: live data retrieval for grounding shall complete without materially slowing the AI response, given the current dataset size (~10–20 records). • Correctness: grounded responses shall be verifiably traceable to the retrieved data supplied in the prompt

Lazizbek Ismoilov

Functional Requirements	Non-functional Requirements
• The system shall allow Admin users to create a new timetable session (course code, session type, day, start time, end time, room, semester) • The system shall allow users to view/read all sessions and filter by course code • The system shall allow Admin users to update an existing session's room, start time, or end time • The system shall allow Admin users to delete a session from the timetable • The system shall automatically detect scheduling clashes (same day, same room, overlapping time range)	• Performance — GET /timetable shall respond within 500ms under normal local Docker execution (validated: average ~2.7ms across 20 sequential requests) • Usability — The UI shall follow the shared team CSS theme (ink/paper/brass tokens) and be usable without training • Reliability — CRUD operations shall complete successfully with appropriate error handling (400 for missing fields, 404 for not-found records) • Maintainability — Code shall follow modular structure (separate frontend, backend, database layers) to support incremental extension in Release 1/2 • Portability — The microservice shall be fully containerised with Docker and run

• The system shall restrict Create/Update/Delete/Clash-check actions to the Admin role; Student and Lecturer roles are read-only (browse, filter, ask AI) • The frontend shall communicate with the backend/API via REST endpoints (GET, POST, PUT, DELETE) to perform CRUD operations • The system shall integrate with AI-Mode to answer natural-language scheduling questions, grounded in the live timetable data retrieved from the database • The system shall display timetable data in the unified frontend UI, following the shared CSS theme and accessible from the unified home page • The system shall demonstrate the Plan → Act → Observe → Adapt agentic workflow when processing an AI-assisted scheduling request • Release 1: The system shall let users call the Timetable MCP tools (check_room_availability, get_sessions_by_course, find_timetable_clashes) from the frontend, through the backend/API, and display the structured result • Release 1: The system shall answer scheduling-policy questions through the shared RAG server and show source citations and a confidence category (high, medium or low)	successfully via docker-compose up alongside other services • Security — Input shall be validated before database writes; error messages shall not expose internal implementation details • Scalability — The database design shall support growth beyond the initial 10 records without requiring schema changes • Availability — The microservice shall be available whenever the Docker Compose stack is running, with no manual intervention needed • Interoperability — The backend/API shall expose REST endpoints (course_code, staff_id fields) in a consistent format so other student microservices (Course Catalogue, Staff Management) can integrate • Testability — The microservice shall support GitHub Actions CI validation via student-3.yml, and passed a 16-check local validation suite covering all endpoints • Security / tool boundaries (R1) — MCP tools shall be read-only, validate all inputs and read data only through the backend API; the backend shall allow-list only the three Timetable tools • Grounding and traceability (R1) — Every RAG answer shall cite the retrieved policy sections; sections below 0.55 cosine similarity shall not be used as evidence

• Release 1: The system shall return an insufficient-context response, without calling the LLM, when no policy section passes the similarity threshold • Release 1: The shared agentic loop shall validate the shared MCP and RAG servers in separate MCP and RAG validation modes	• Reliability (R1) — If the MCP or RAG server is unavailable, the backend shall return a clear 503 and CRUD and AI-Mode shall keep working • Interoperability (R1) — The backend shall reach the host-run MCP (port 8000) and RAG (port 5050) servers through host.docker.internal, configured by environment variables • Testability (R1) — MCP and RAG shall be disabled in CI (MCP_ENABLED/RAG_ENABLED=false) and covered by mocked unit tests; validate.sh passed 27/27 checks locally

Yi Zhang

Functional Requirements	Non-functional Requirements
• The system should support CRUD operations for courses and enrolments. • The system should provide AI Mode using current course and enrolment data. • The system should provide MCP Mode through the shared local MCP server. • MCP Mode should support structured queries about course and enrolment data. • The system should provide RAG Mode through the shared local RAG server. • RAG responses should use retrieved context and provide grounded answers with citations. • The system should return an insufficient-context response when relevant information is unavailable.	• The feature should use separate frontend, backend/API, and database microservices. • Docker should be used to containerise the Student 4 microservices. • User input and MCP requests should be validated before processing. • MCP and RAG servers should run locally and remain non-containerised. • Normal UI and AI Mode should continue working if MCP or RAG is unavailable. • GitHub Actions should validate the Student 4 services and MCP/RAG integration files. • The system should return suitable HTTP error responses when requests fail.

Vu Tien Thanh Nguyen

Functional Requirements	Non-functional Requirements
• The system shall display all assessment records. • The system shall allow users to search and filter assessments by course, assessment name, and assessment type. • The system shall allow users to retrieve individual assessment details. • The system shall allow assessments to be created, updated, and deleted. • The system shall display grade records. • The system shall allow grades to be retrieved by student ID and course. • The system shall allow grade records to be created, updated, and deleted. • The system shall validate student IDs through the Student Records service before creating or updating grade records. • The system shall retrieve course information from the Course Management service. • The system shall provide an AI Mode for natural-language questions about assessments and grades. • The AI shall use current assessment and grade information before answering questions.	• **Usability:** The Assessment & Grades interface shall follow the shared team design and provide clear interfaces for viewing and managing assessment and grade records. • **Reliability:** CRUD operations shall return suitable HTTP responses and error handling when records cannot be found, input is invalid, or another required service is unavailable. • **Maintainability:** The feature shall use a modular microservice structure with separate frontend, backend/API, and database services. • **Portability:** The Assessment & Grades feature shall be containerised using Docker and operate as part of the shared Docker Compose application. • **Security:** User input shall be validated before assessment or grade information is written to the database. • **Data Integrity:** Grade records shall reference valid assessment records, and student IDs shall be validated through the Student Records service before grade creation or update. • **Availability:** Standard assessment and grade CRUD functionality shall remain separate from AI Mode so that normal operations can continue if Ollama is unavailable. • **Interoperability:** The feature shall communicate with other microservices using REST APIs and the shared Docker network. • **AI Reliability:** AI responses shall be grounded in retrieved assessment and grade information, and the model shall be instructed not to invent unsupported information. • **Testability:** Backend endpoints shall be validated using automated tests and CI/CD checks. • **Persistence:** Assessment and grade data shall be stored in SQLite using a named Docker volume so that data can persist across container restarts.

For Release 1, the Assessment & Grades Management feature also includes the following functional requirements: • The system shall provide MCP Mode for read-only queries about assessment and grade information. • The system shall provide a list_course_assessments MCP tool to retrieve assessments, due dates, weights, and total assessment weight for a course. • The system shall provide a get_student_grade_summary MCP tool to retrieve a student's grade summary. • The system shall provide a get_upcoming_assessments MCP tool to retrieve upcoming assessments based on a date and result limit. • The system shall validate MCP tool inputs before accessing assessment data. • The system shall restrict the Assessment Service to the registered Student 5 MCP tools. • The system shall provide RAG Mode for retrieval-grounded questions about assessments and course-level grade information. • The system shall retrieve relevant assessment records, anonymised course-level grade statistics, and assessment policy	Release 1 non-functional requirements include: • Security: MCP tools shall be read-only and shall not modify assessment or grade records. • Validation: Invalid MCP inputs shall be rejected before assessment data is accessed. • Reliability: RAG responses shall be grounded in retrieved evidence rather than unsupported model knowledge. • Privacy: Individual student marks shall not be included in the RAG knowledge base. • Availability: Normal Assessment & Grades functionality shall remain available when MCP or RAG services are disabled. • Interoperability: The Assessment Service shall communicate with the shared MCP and RAG servers through the configured service URLs. • Testability: MCP tools, RAG behaviour, and the corresponding backend routes shall be covered by automated tests.

information for RAG questions. • The system shall provide citations and source information for RAG responses. • The system shall provide a confidence level for RAG responses. • The system shall return insufficient evidence when the retrieved information does not support the user's question. • The RAG knowledge base shall not contain individual student marks.	

2.3 Risk Management Plan (i)

Rigel Rivamonte

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	The database service may be unavailable when the application service attempts to retrieve student records.	M	H	Use Docker Compose dependencies, service names, and request timeouts.	Return a clear 503 service-unavailable response to the frontend.
2	Ollama may not be running or the required model	M	M	Configure the Ollama URL and model through environment variables and test AI Mode locally.	Keep deterministic answers and standard

	may not be installed.				CRUD/search functions available.
3	Cross-origin frontend requests may fail because the frontend and Flask service use different ports.	M	H	Enable Flask-CORS in the enrolment/application service.	Review CORS settings and return readable frontend error messages.
4	Invalid or incomplete student data may be sent during record creation.	M	M	Validate required fields before creating a record.	Return an HTTP 400 response explaining which fields are missing.
5	Duplicate or inconsistent student IDs may be created.	L	M	Use the SQLite primary key and generate new IDs from the highest existing student number.	Correct the record manually and reseed the development database if required.
6	SQLite data may be lost when a container is recreated.	L	H	Use the named Docker volume student1_database_data.	Rebuild and reseed the database using init_db.py.
7	Docker build paths may fail during group integration.	M	M	Test each Dockerfile using the same build context configured in Docker Compose.	Build services individually to identify and correct the failing Docker path.
8	Merge conflicts may occur in the shared docker-compose.yml.	M	M	Keep feature code inside student-1 and make small focused commits.	Resolve the shared compose file manually while preserving all team service definitions.
9	AI answers may contain unsupported details if the	M	M	Pass live records into the prompt and explicitly instruct the model not to invent information.	Return a deterministic answer where supported or

	prompt is too broad.				state that the data is unavailable.

Jeriko Arceo

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Dependency on Course data owned by another student — schema or API mismatch could break staff-course assignments	H	H	Agree early with the Course feature owner on a shared schema/API contract (e.g. course_id format, whether to call their API or share a table)	If integration isn't ready in time, use a local mock/stub courses table with sample data so Staff Management can still be demoed independently
2	AI-Mode / Ollama integration fails or produces unreliable responses for staff/course queries	M	M	Test AI-Mode early with simple prompts; keep query scope narrow (e.g. one or two supported query types)	Fall back to standard UI search/filter if AI response fails, so core CRUD still works
3	Docker networking/integration issues when combining with teammates' microservices in shared docker-compose.yml	M	H	Test the microservice in isolation first, then integrate early and often rather than waiting until the deadline	Keep a working standalone Docker Compose version as backup for individual demonstration
4	Data integrity issues in the staff-course assignment table (e.g. duplicate or orphaned assignments)	M	M	Add validation checks and foreign key constraints in the database schema	Add a manual data-cleanup script to reset/reseed data before demos

5	Time constraints — assignment feature (many-to-many relationship) takes longer to build than standard CRUD	M	M	Prioritise core staff CRUD first (must-have), treat assignment feature as a stretch goal within Release 0 sprint	If time runs out, submit staff CRUD complete and document assignment feature as a known limitation
6	GitHub Actions workflow (student-x.yml) fails due to environment/config errors	L	M	Test workflow file locally and in a feature branch before merging to main	Debug using workflow logs; keep last known working workflow version to revert to
7	Merge conflicts with teammates when integrating into shared repository	M	L	Commit and push regularly in small increments; communicate with team before major structural changes	Resolve conflicts collaboratively; use feature branches and pull requests rather than direct commits to main
8	The LLM selects the wrong MCP tool, or calls it with invalid/missing arguments, producing an incorrect or failed response.	M	M	Define clear, narrowly-scoped tool schemas with input validation; test each tool call in isolation before wiring it into the AI flow.	Fall back to the existing /ask-with-context context-injection approach if MCP tool-calling proves unreliable with qwen2.5:0.5b
9	Retrieval (RAG/context) returns no relevant data, or stale data, if database-service is unavailable — causing the AI to answer without proper grounding.	M	M	Explicitly check for an empty or failed retrieval result before building the prompt; clearly flag when live data couldn't be fetched.	Return a deterministic "data unavailable" message instead of letting the LLM answer ungrounded.

Yi Zhang

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Shared MCP server may be unavailable.	M	H	Check the MCP connection and server URL before use.	Show a clear error and keep Normal UI and AI Mode available.
2	Invalid MCP tool requests may cause errors.	M	H	Validate tool names and inputs before sending requests.	Reject invalid requests and return a clear error message.
3	RAG may return irrelevant or insufficient context.	M	H	Check retrieved context before generating an answer.	Return an insufficient-context response instead of guessing.
4	RAG or Ollama may be unavailable.	M	M	Configure service URLs correctly and test the services locally.	Keep normal Course and Enrollment functions available.
5	Docker or integration conflicts may prevent Student 4 services from connecting correctly.	M	H	Use consistent service names, ports, and Docker network settings.	Check the configuration, rebuild the affected services, and test again.

Lazizbek Ismoilov

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Flask backend defaults to binding 127.0.0.1, which is unreachable once containerised	M	H	Explicitly bind host="0.0.0.0" in app.run() before containerising	Caught via standalone Docker testing before integration; documented in evidence log

2	Frontend and backend run as separate origins (different ports) once split into microservices, causing CORS/cross-origin request failures	M	H	Enable flask-cors on the backend early, before building frontend features that call it	Add CORS headers manually via after_request if flask-cors is unavailable
3	htmx 2.x blocks cross-origin AJAX requests by default (selfRequestsOnly), causing silent frontend failures	L	M	Test each fetch/AJAX call against the real cross-origin setup, not just same-origin during development	Replace htmx.ajax() calls with plain fetch() for cross-origin requests
4	Docker container name conflicts when switching between standalone and shared docker-compose runs	M	L	Use docker compose down (not just Ctrl+C) when switching contexts between standalone and shared stacks	Force-remove conflicting containers by name (docker rm -f) before retrying
5	Merge conflicts in shared docker-compose.yml when multiple students' services are integrated close together	H	M	Commit and push service additions in small increments; communicate before major structural changes	Resolve conflicts by merging both services' blocks manually rather than picking one side
6	Dependency on Staff Management's staff_id format for the staff_id foreign key field	M	L	Keep staff_id as a nullable generic TEXT field until format is confirmed with the feature owner	Ship Release 0 with staff_id unpopulated; document as a known integration point for Release 1
7	Time constraints — building a fully role-based UI (Student/Lecturer/Admin) alongside CRUD and AI-	M	M	Prioritise core CRUD and clash detection first (must-have), treat full role-based view	If time runs out, ship with a simpler role toggle and document remaining polish

	mode takes longer than a flat single-role UI			polish as a stretch goal within the Release 0 sprint	as a known limitation
8	Shared MCP or RAG server is not running when the Timetable backend calls it	M	H	Return a clear 503 naming the server URL; keep CRUD and AI-Mode independent of MCP/RAG	Start both servers on the host and rerun validate.sh
9	Host port collision once RAG runs outside Docker (RAG and Timetable backend both on 5003)	H	M	Agree a team port contract before coding: MCP 8000, RAG 5050	Change the RAG PORT environment variable and update backend URLs
10	Small local LLM (qwen2.5:0.5b) gives unsupported or poorly worded RAG answers	M	M	Pass only retrieved policy sections as evidence, require citations, abstain below the similarity threshold	Citations let users verify; tune thresholds or use a larger model
11	A Python dependency missing from the container image is only found at runtime	M	H	Verify backend imports in a clean environment with only requirements.txt	Diagnose with docker logs, fix requirements.txt and rebuild

Vu Tien Thanh Nguyen

Risk #	Risk	Likelihood	Impact	Mitigation	Contingency
1	Student Records service is unavailable when validating a grade's student_id	M	H	Use REST API communication and appropriate exception handling when	Return a clear 503 response and prevent an invalid/unverified grade from being recorded

				validating students	
2	Course Management service is unavailable	M	M	Handle failed course lookups gracefully	Display the course ID without the course name until the service becomes available
3	Ollama or Qwen model is unavailable	M	M	Keep normal CRUD functionality independent from AI Mode	Return a clear AI service error while keeping Assessment & Grades CRUD available
4	Invalid assessment or grade data is submitted	M	M	Validate required fields, numeric fields, student ID format and referenced assessment IDs	Return appropriate 400/404 responses without saving invalid records
5	Docker services fail to communicate	M	H	Use the shared enrolment-network, service names and environment variables for service discovery	Check container status, ports, network configuration and service URLs
6	Merge conflicts occur during integration	M	M	Keep Student 5 code within the student-5/ folder and integrate shared files carefully	Resolve conflicts manually while retaining required service configurations
7	AI generates unsupported assessment or grade information	M	H	Supply retrieved database evidence to the model and instruct it not to invent data	Return that information is unavailable when the retrieved evidence does not contain the answer

8	Changes introduce errors into the Student 5 services	M	M	Use automated tests and GitHub Actions validation	Fix failed tests/builds before merging changes
9	Shared MCP server is unavailable	M	H	Use environment variables and service availability checks	Return an appropriate service error while keeping normal Assessment & Grades functionality available
10	Shared RAG server is unavailable	M	H	Keep RAG functionality separate from normal CRUD operations and validate the RAG route	Return an appropriate error or insufficient-service response while keeping normal functionality available
11	RAG generates unsupported information	M	H	Ground responses using retrieved assessment evidence and require citations	Return insufficient evidence rather than unsupported information
12	Invalid MCP tool input is provided	M	M	Validate course codes, student IDs, dates and limits before processing	Reject the request without accessing or modifying data
13	MCP exposes another feature's tool	L	H	Use an explicit Student 5 tool allowlist	Reject calls to tools outside the Student 5 tool set
14	Individual student marks are exposed through RAG	L	H	Exclude individual student marks from the RAG knowledge base	Use the appropriate MCP tool for individual grade information

2.4 Data Design

The University Management System uses a distributed data design. Each feature manages its own data through a separate database service, while the different microservices communicate through REST APIs.

The main data areas include student records, staff records, course and enrolment data, timetable information, and assessment and grade information.

2.4.1 Conceptual Data Model

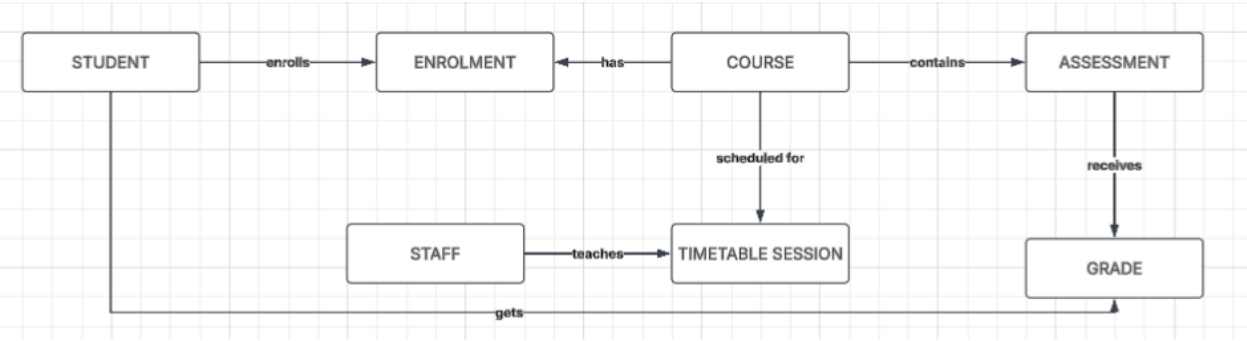
The conceptual data model identifies the main entities in the system and their relationships.

The main entities include:

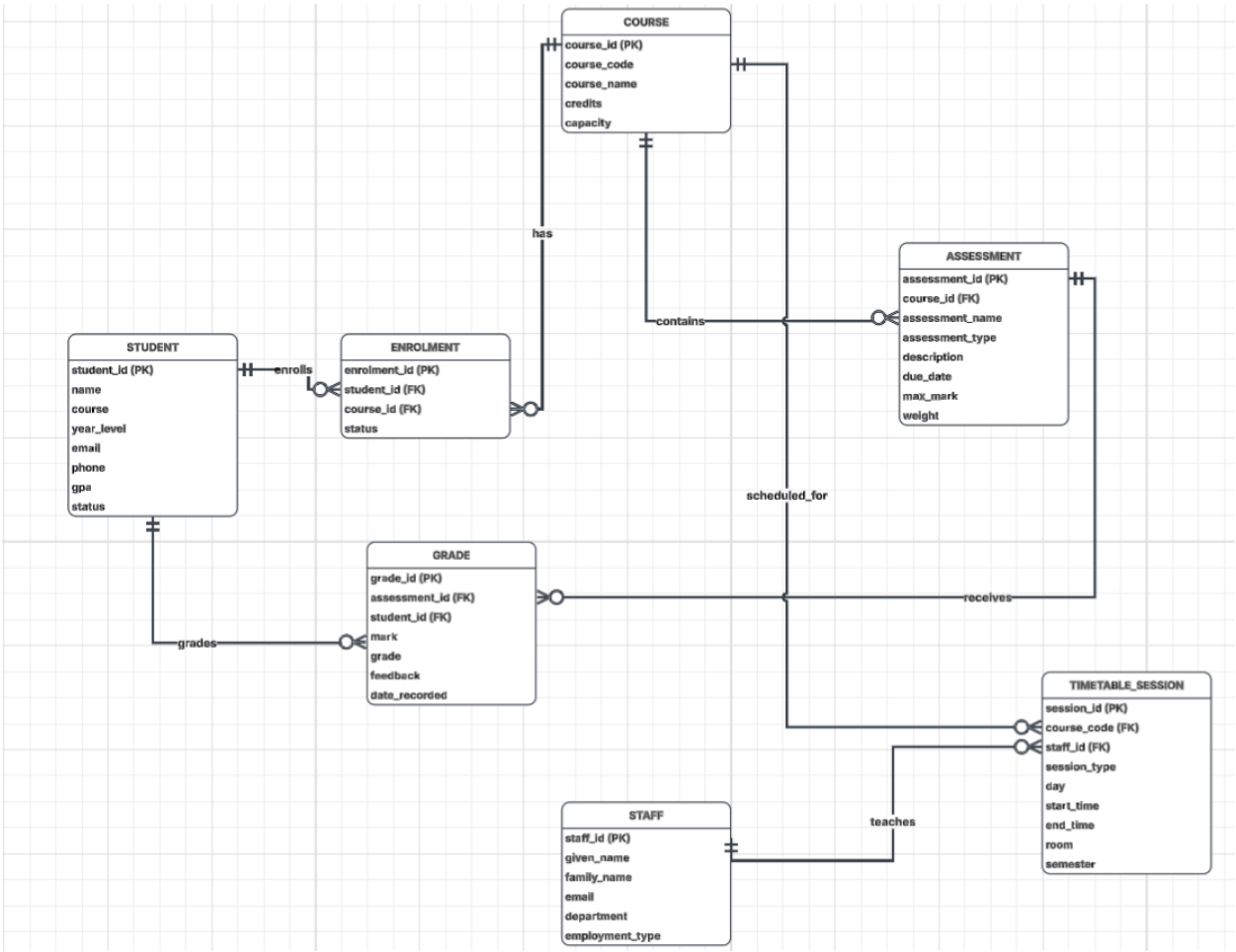
- Student
- Staff
- Course
- Enrolment
- Timetable Session
- Assessment
- Grade

Students can enrol in courses, courses can have timetable sessions and assessments, and staff can be linked to teaching activities. Grades are used to record student results for assessments.

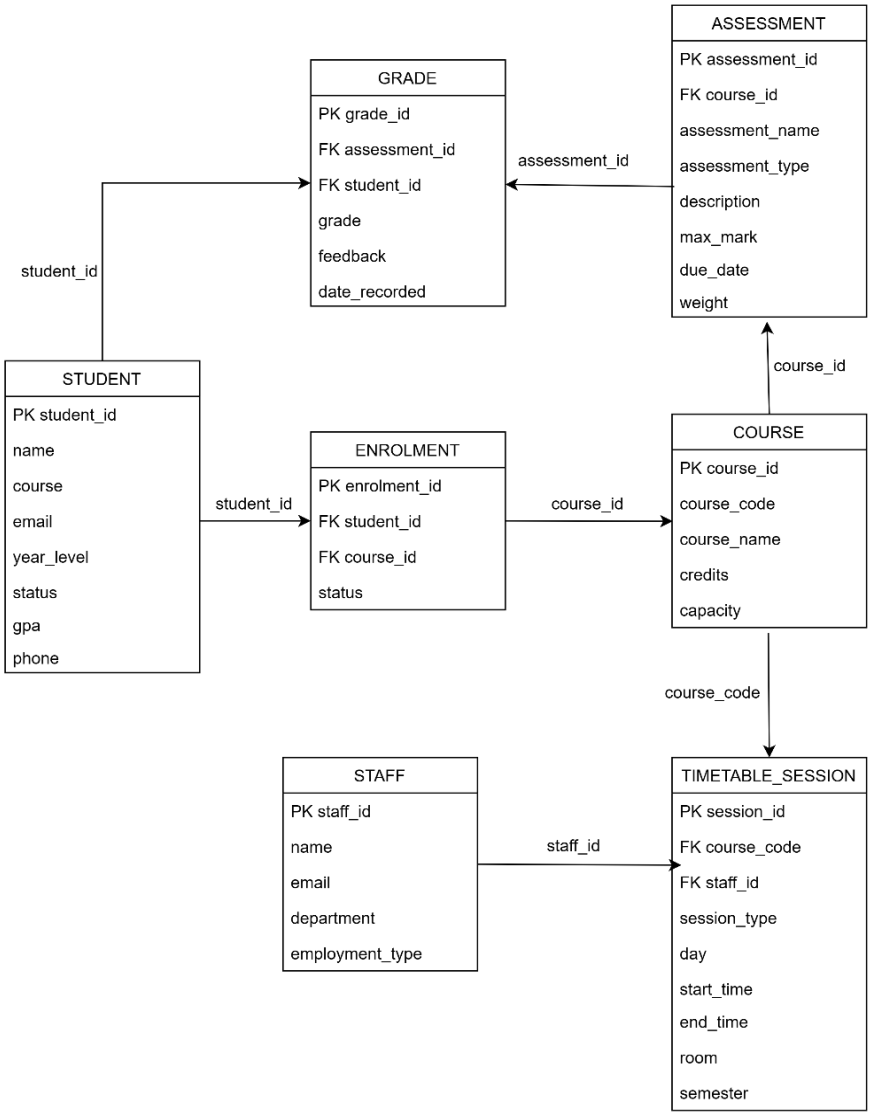
These relationships provide the basic data structure for the integrated University Management System.



2.4.2 ERD


COURSE
course_id (PK)
course_code
course_name
credits
capacity
has
contains
ASSESSMENT
assessment_id (PK)
course_id (FK)
assessment_name
assessment_type
description
due_date
max_mark
weight
STUDENT
student_id (PK)
name
course
year_level
email
phone
gpa
status
enrolls
ENROLMENT
enrolment_id (PK)
student_id (FK)
course_id (FK)
status
scheduled_for
GRADE
grade_id (PK)
assessment_id (FK)
student_id (FK)
mark
grade
feedback
date_recorded
receives
grades
TIMETABLE_SESSION
session_id (PK)
course_code (FK)
staff_id (FK)
session_type
day
start_time
end_time
room
semester
STAFF
staff_id (PK)
given_name
family_name
email
department
employment_type
teaches

2.4.3 Logical Data Model



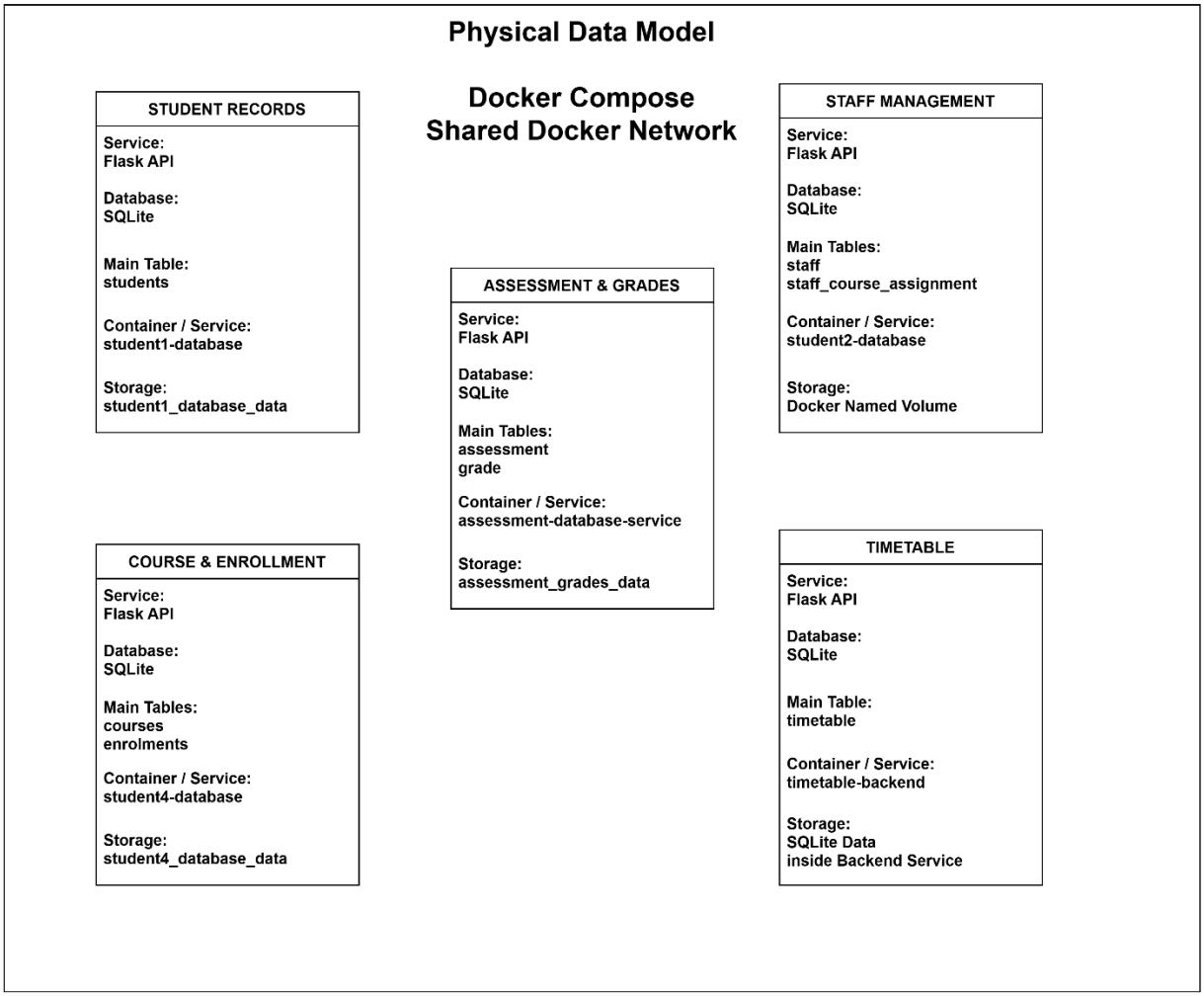
2.4.4 Physical Data Model

The physical data model shows how the system data is implemented in the final application. Each feature uses a separate service and database structure, with SQLite used for data storage.

The Student Records, Staff Management, Course and Enrollment, Timetable, and Assessment and Grades features are deployed as separate Docker services. The services run together through the shared docker-compose.yml and shared Docker network.

Each database service manages its own data, and persistent storage is provided through Docker volumes where required. Cross-feature communication is handled through REST APIs instead of directly accessing another service's database.

This design keeps the system modular and allows each feature to be developed, tested, and integrated independently.

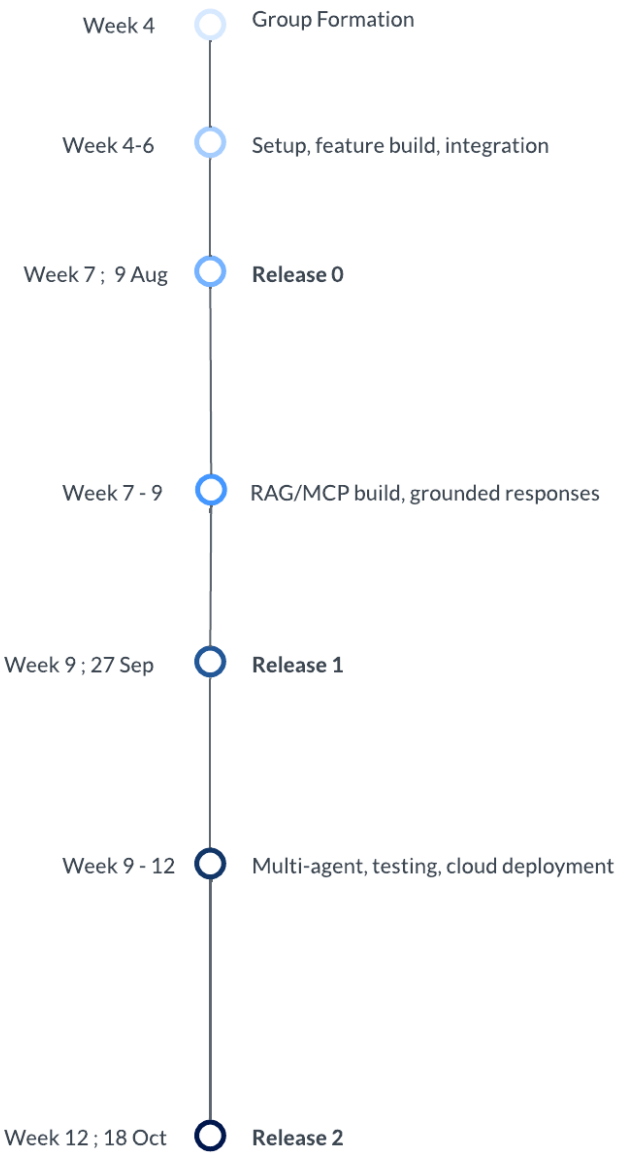


2.5Overall Project Plan (Timeline/Gantt/Milestones)

Timeline

Project Timeline

University Management Application



Gantt Chart

	W4	W5	W6	W7	W8	W9	W10	W11	W12
Team Formation									
Setup									
Feature Build									
AI Integration									
MCP + RAG build									
Multi-agent, cloud									



Milestones

	Milestone	Week	Date	Description
1	Team formation & topic approval	4		Group finalised, Canvas group formed, topic and individual features approved by tutor
2	Repository & environment setup complete	5		Shared GitHub repo created with standard structure; Docker, Ollama, and dev environments configured for all members
3	Individual CRUD functionality complete	6		Each student's frontend/backend/database microservice supports Create, Read, Update, Delete independently
4	AI-Mode integration complete	6-7		Each microservice successfully calls Ollama + approved LLM
5	Release 0 submission	7	9 Aug	Integrated app, Docker Compose, GitHub Actions workflows, technical report, and showcase video submitted

6	MCP & RAG servers integrated	8		Shared MCP and RAG servers built and connected to individual microservices
7	Grounded AI responses functional	8-9		AI responses demonstrably use retrieved context (RAG) across the integrated app
8	Release 1 submission	9	27 Sep	Updated integrated app, updated report, and showcase video submitted
9	Multi-Agent System integrated	10-11		Planner, Worker, and Reviewer agents functioning within the shared workflow
10	Pre/post-testing complete	11		Unit, integration, and system testing evidence collected for all microservices
11	Cloud deployment successful	11-12		Integrated app deployed to Azure or AWS with AI-Mode enabled, MCP/RAG/Multi-Agent disabled
12	Release 2 submission	12	18 Oct	Final integrated app, complete report, and showcase video submitted

3.0 Repository Structure

3.1 Shared GitHub Repository Layout

The repository (ASD-Project-Group18) is organised, so each student's feature is fully self-contained, with shared, team-wide resources kept separate from individual work.

Top-level layout:

- .github/workflows/ — one CI/CD workflow file per student (e.g. student-3.yml), each scoped to only trigger on changes to that student's own folder
- shared/ — the unified home page (homepage.html), shared CSS theme, and its Dockerfile
- student-1/ through student-5/ — one folder per student, each containing that student's frontend, backend/API, and database code
- docker-compose.yml — the single shared file orchestrating all services across all five features plus the shared home page
- README.md — project overview and per-feature setup instructions

- ai-services/mcp-server/
- ai-services/rag-server/

Each student-N/ folder follows a three-tier internal structure, though the exact naming varies slightly by student (e.g. frontend/backend/database for Student 3 and Student 5, versus frontend-service/enrolment-service/database-service for Students 1, 2, and 4) — reflecting that folder-level naming was standardised to student-1…student-5 as a group during Release 0, while internal sub-folder naming was left to each student's own convention.

3.2 Branching Strategy and Folder Organization

Each student develops on their own feature branch (e.g. student-3/timetable-integration, jeriko, Yi-Zhang, rigel) and merges into main via Pull Request, following the spec's required PR-based workflow rather than direct commits to main.

Folder organisation directly enforces the microservice boundary rule: a student's code only ever lives inside their own student-N/ folder, and cross-feature data is accessed only via that feature's REST API, never by reading another student's database file or importing their code directly.

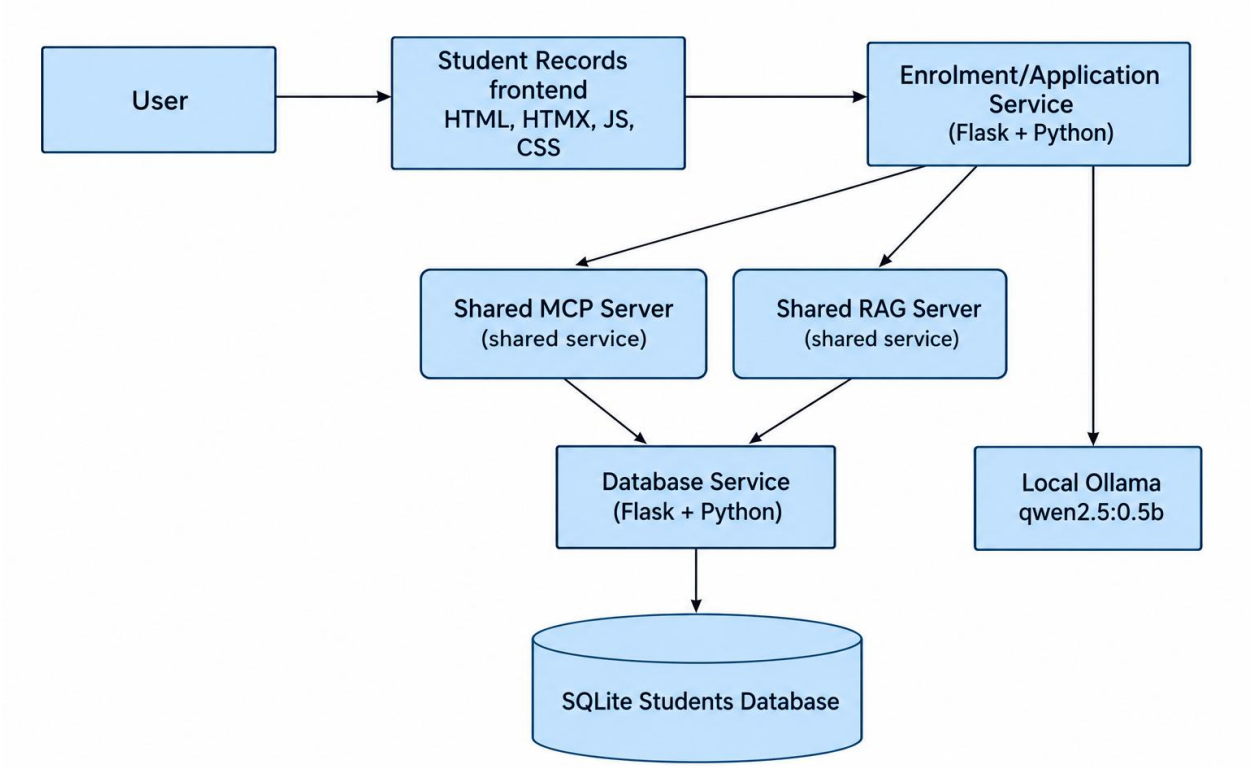
During Release 0, the team encountered and resolved two structural issues through this branching workflow:

- Inconsistent top-level naming (student1, Student2, Student4, student-5, student-3) was standardised to student-1 through student-5 in a coordinated update to main
- A real merge conflict occurred in the shared docker-compose.yml when two students' service additions landed close together; it was resolved by merging both services' blocks rather than discarding either side's work

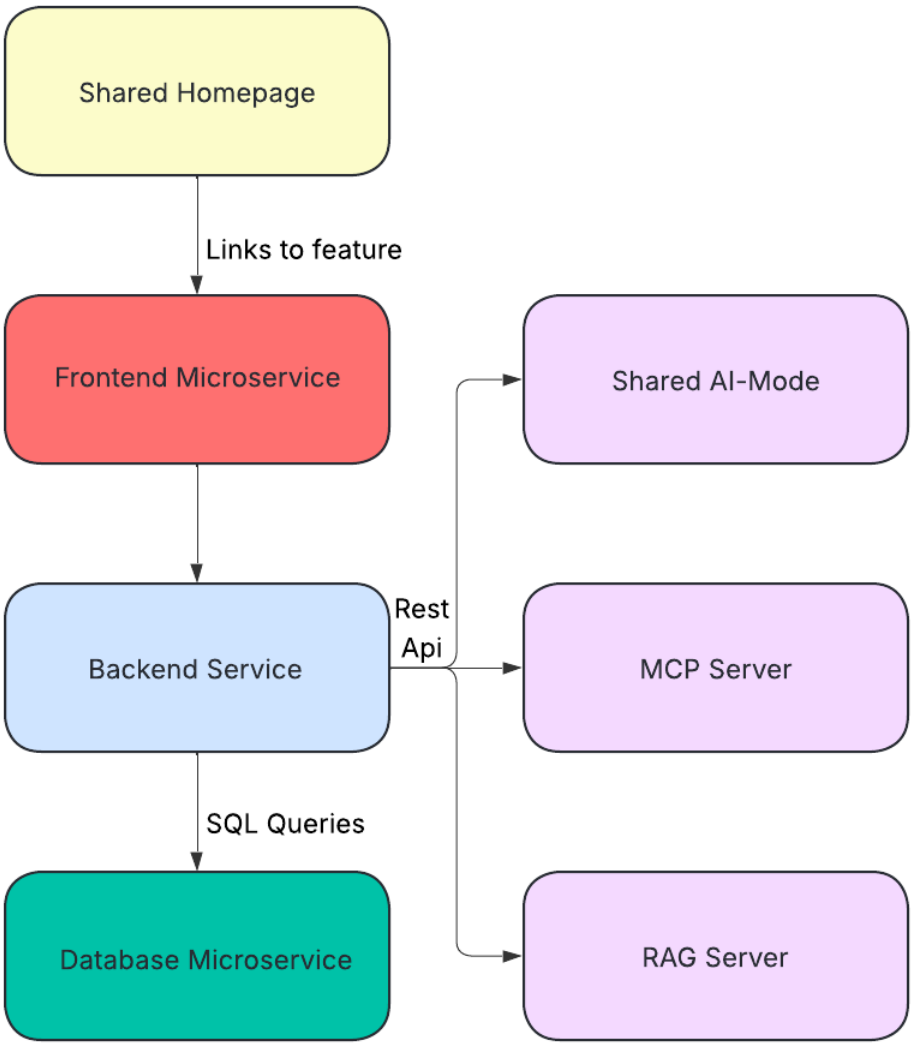
Commits are made in small, regular increments and pushed early to reduce the size and difficulty of merge conflicts, consistent with the mitigation strategy documented in each student's Risk Management Plan (Section 2.4).

4.0 Individual Software Architecture Diagrams (Frontend, Backend/API, Database)

4.1 Student 1



4.2 Student 2 – Jeriko Arceo



4.3 Student 3 – Lazizbek Ismoilov

The Timetable & Class Scheduling feature uses a microservice architecture consisting of a Frontend Microservice, Backend/API Microservice, SQLite Database, and a local AI-Mode agent.

Release 1 extension. The backend adds GET /api/timetable (JSON read by the MCP tools), /mcp/tools and /mcp/call, which act as an MCP client to the shared MCP server over streamable HTTP, and /rag/answer, which posts the question with domain “timetable” to the shared RAG server. The frontend adds MCP Tools and Policy Q&A tabs. Both shared servers run on the host

outside Docker; the backend container reaches them at host.docker.internal:8000/mcp and host.docker.internal:5050.

Timetable MCP tools (ai-services/mcp-server/timetable_tools.py):

- check_room_availability(room, day, start_time, end_time) → available, conflicts[]
- get_sessions_by_course(course_code) → session_count, sessions[]
- find_timetable_clashes() → clash_count, clashes[]

Tool boundaries: all three tools are read-only, validate their inputs (invalid input returns a ToolError) and read data only through the backend API, never the SQLite file. The backend allow-lists these three tool names and the shared server rejects unregistered tools.

Timetable RAG domain (ai-services/rag-server/timetable_rag.py). Knowledge source: corpus/timetable_policy.md (Room Booking & Scheduling Policy, 12 sections). Each section is embedded with nomic-embed-text and ranked by cosine similarity. Sections scoring 0.55 or above are passed to qwen2.5:0.5b as the only evidence; confidence is high (≥ 0.70), medium (0.55–0.70) or low. On low confidence the server returns an insufficient-context response and the LLM is not called. Citations are built from the retrieved sections, not from the model's text.

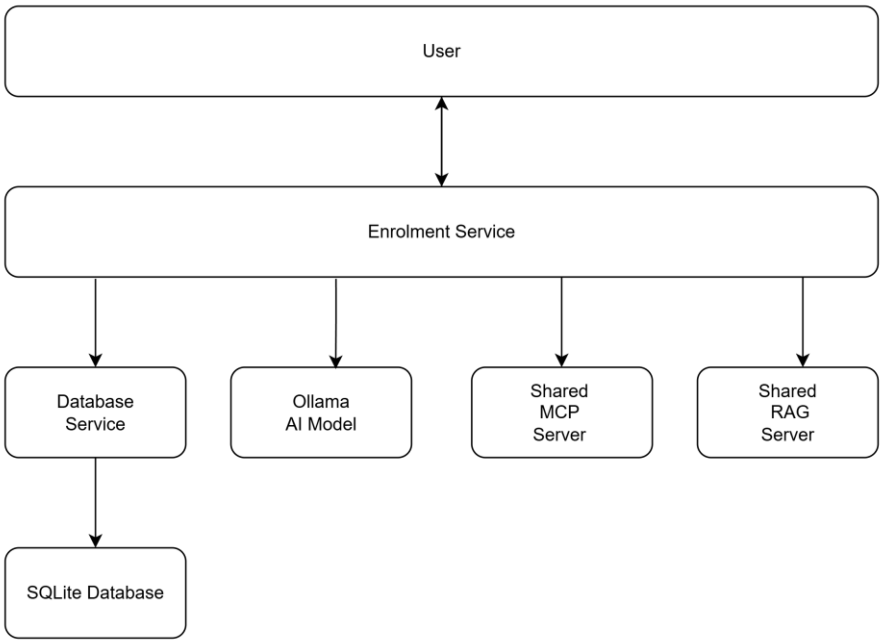
4.4 Student 4 Yi Zhang

The Course and Enrollment Management feature uses a microservice architecture. The main components are the Frontend Service, Enrolment Service, Database Service, and local Ollama AI service.

The Frontend Service provides the user interface for Course Management, Student Enrolment, AI Mode, MCP Mode, and RAG Mode. The Flask Enrolment Service handles frontend requests and communicates with the Database Service, shared MCP Server, and shared RAG Server.

The Database Service manages course and enrolment data in SQLite. AI Mode uses local Ollama, while MCP Mode and RAG Mode connect to the shared local MCP and RAG servers.

The basic architecture is:



This structure separates the frontend, application logic, database, and AI services. It also allows the Student 4 feature to integrate with the shared Release 1 services.

4.5 Student 5

The Assessment & Grades Management feature uses a three-service microservice architecture consisting of a Frontend Service, Assessment Service, and Assessment Database Service.

The **Frontend Service** provides the user interface for managing assessments and grades. It is implemented using HTML, CSS, and JavaScript and is served through Nginx. The frontend is available on port **8085** and communicates with the Assessment Service through REST API requests.

The **Assessment Service** is implemented using Flask and acts as the main application layer. It provides the assessment and grade CRUD endpoints, input validation, cross-service integration, error handling, and AI Mode. The service runs on port **5021**.

The **Assessment Database Service** is implemented using Flask and SQLite and runs on port **5022**. It stores the assessment and grade records and provides database operations to the Assessment Service. A named Docker volume, assessment_grades_data, is used to persist the SQLite data.

The Assessment Service also communicates with the **Student Records service (Student 1)** to validate student IDs and retrieve student information. It communicates with the **Course Management service (Student 4)** to retrieve course information.

For AI Mode, the Assessment Service communicates with the local **Ollama** service using the qwen2.5:0.5b model. Relevant assessment and grade information is retrieved before the AI response is generated.

The basic architecture is:

User → Assessment & Grades Frontend → Assessment Service → Assessment Database Service

The Assessment Service also communicates with:

Assessment Service → Student 1 Student Records Service

Assessment Service → Student 4 Course Management Service

Assessment Service → Ollama AI Service

For Release 1, the Assessment Service was extended to communicate with the shared MCP and RAG services.

For MCP Mode, the frontend sends a request to the Assessment Service, which communicates with the shared MCP server. The MCP server executes the registered Student 5 read-only tools and retrieves the required assessment information from the Assessment database.

For RAG Mode, the frontend sends a question to the Assessment Service. The Assessment Service communicates with the shared RAG server, which retrieves relevant assessment records, anonymised course-level grade statistics, and assessment policy information before generating a grounded response.

The Release 1 request paths are:

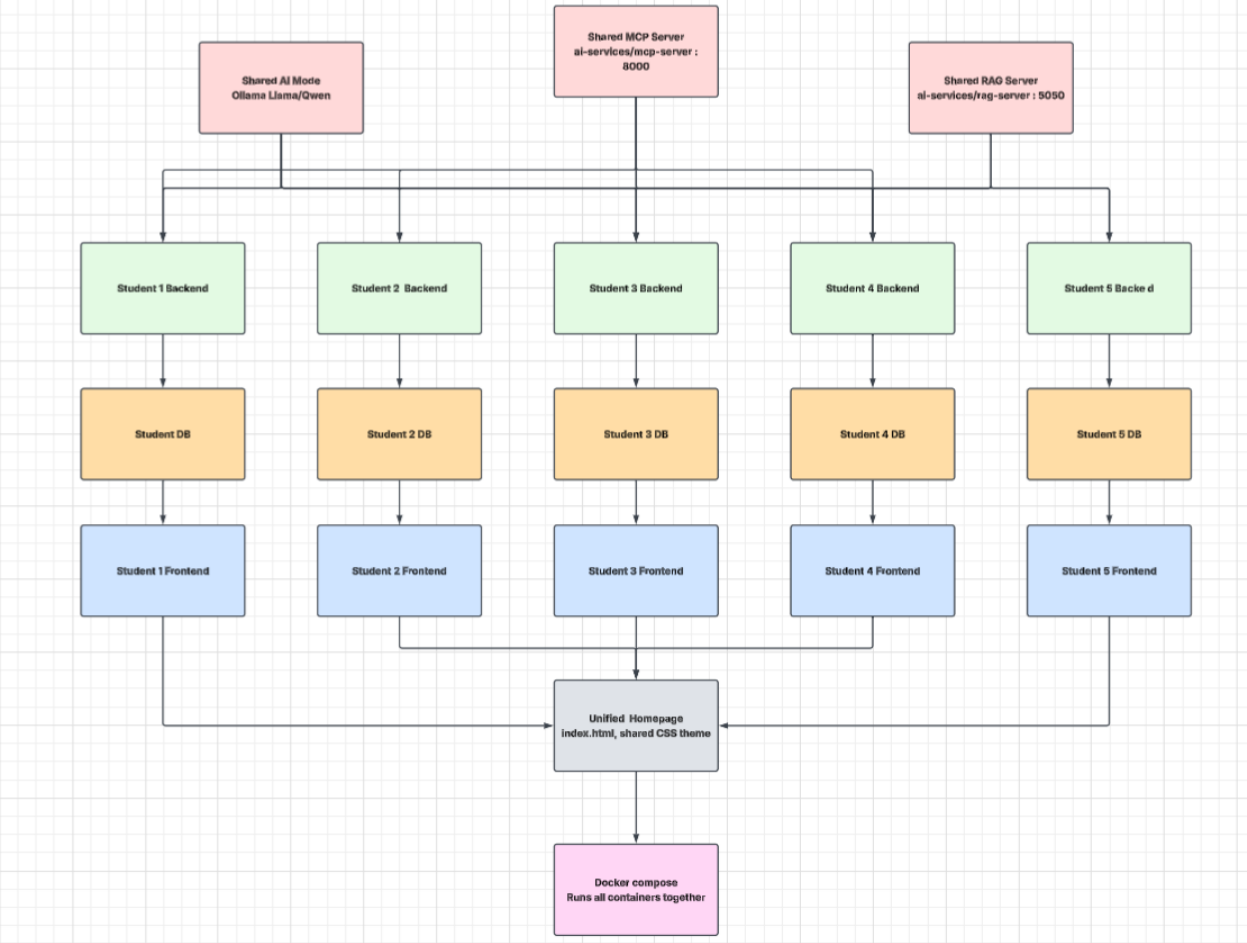
User → Assessment & Grades Frontend → Assessment Service → Shared MCP Server → Assessment Database

User → Assessment & Grades Frontend → Assessment Service → Shared RAG Server → Ollama

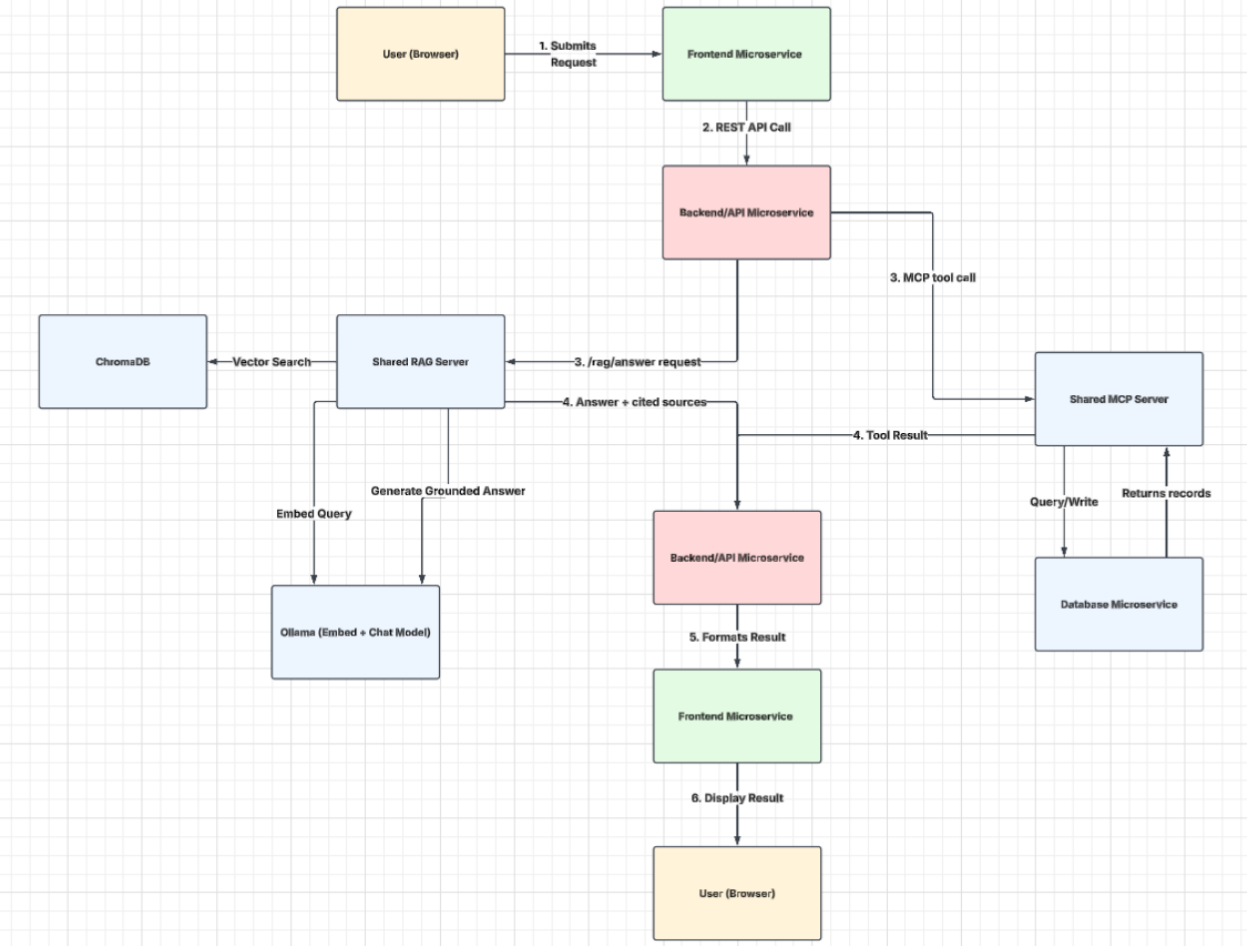
The MCP and RAG servers are shared project-level services and are not deployed as additional Student 5 containers.

5.0 Integrated Software Architecture

5.1 Release 1 Integrated Architecture Diagram



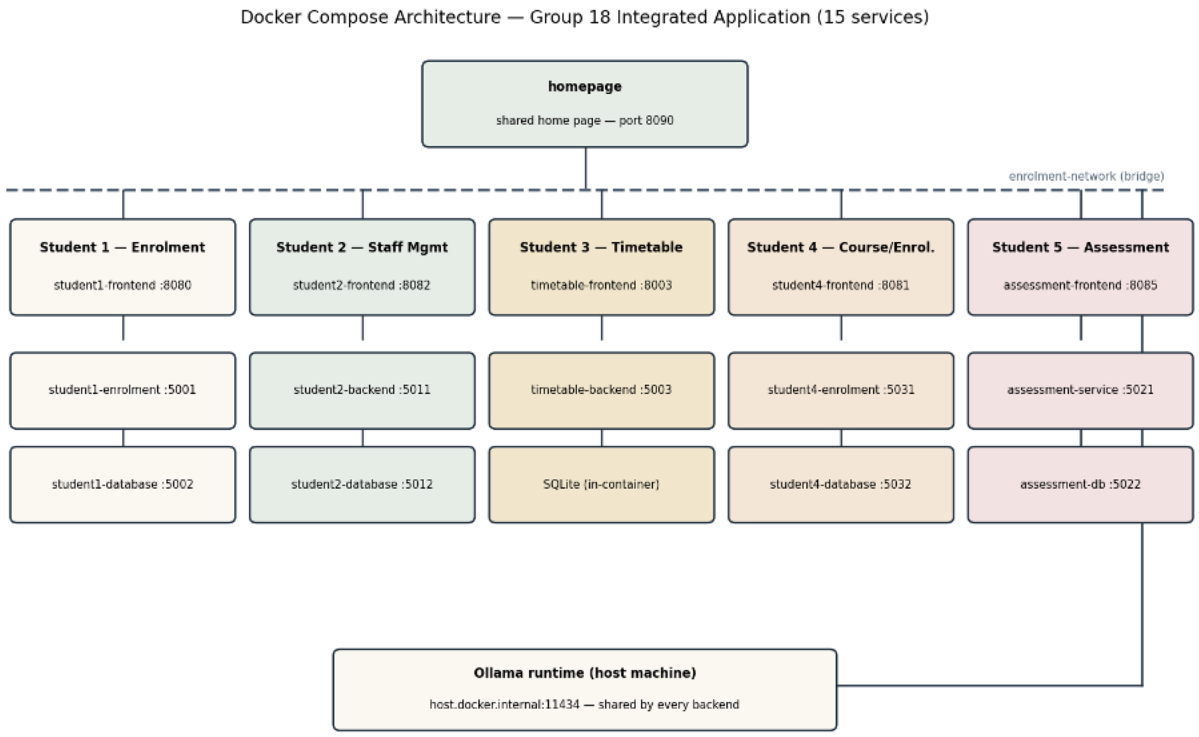
5.2 Component Interaction Description



6.0 Docker Compose Architecture

6.1 Docker Compose Architecture Diagram

The full application runs as 15 containers defined in a single shared docker-compose.yml at the repository root, all connected via one Docker bridge network (enrolment-network) so any service can reach any other by its service name.



6.2 Service/Container Descriptions

All 15 services, their role, and their externally published port:

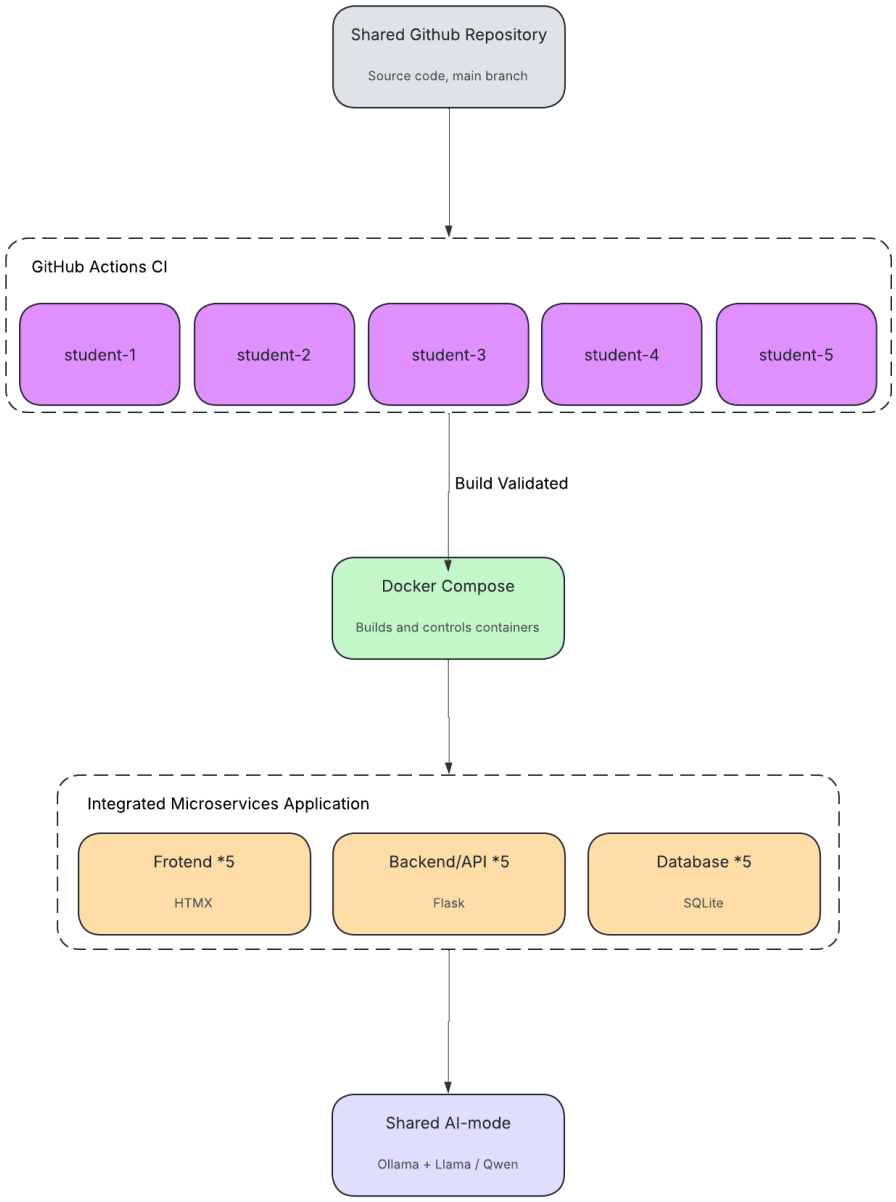
Service (container name)	Purpose & Port
homepage	Shared home page (nginx) linking to all five features — port 8090
student1-frontend	Student 1 (Enrolment) frontend — port 8080
student1-enrolment	Student 1 backend/API (Flask) — port 5001
student1-database	Student 1 database service — port 5002
student2-frontend	Student 2 (Staff Management) frontend — port 8082
student2-backend	Student 2 backend/API (Flask) — port 5011
student2-database	Student 2 database service — port 5012
timetable-frontend	Student 3 (Timetable & Class Scheduling) frontend — port 8003
timetable-backend	Student 3 backend/API (Flask), owns the SQLite timetable table directly (no separate database container) — port 5003
student4-frontend	Student 4 (Course/Enrolment) frontend — port 8081
student4-enrolment	Student 4 backend/API (Flask) — port 5031

student4-database	Student 4 database service — port 5032
assessment-frontend-service	Student 5 (Assessment/Grades) frontend — port 8085
assessment-service	Student 5 backend/API (Flask) — port 5021
assessment-database-service	Student 5 database service — port 5022

Every backend service shares the same host.docker.internal:11434 connection to reach Ollama, which runs on the host machine rather than inside a container. Each feature's database uses a named Docker volume for persistence (e.g. student1_database_data, assessment_grades_data) so data survives container restarts. Getting all 15 services running together required resolving three integration issues: inconsistent folder naming across students, docker-compose.yml paths left pointing at old pre-rename folder locations, and container name conflicts between standalone per-feature testing and the shared stack.

7.0 DevOps Pipeline Architecture

7.1 Overall DevOps Architecture Diagram (GitHub, Actions, Compose, microservices, AI-Mode)



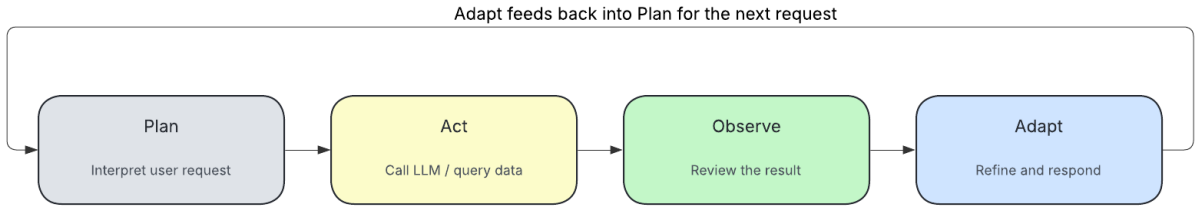
7.2 Pipeline Stage Descriptions

Stage	Description
1. Source code commit	Team members commit and push code changes to their individual feature branches, then merge into the shared GitHub repository via pull requests. This is the entry point that triggers the pipeline.
2. GitHub Actions CI	Each push or pull request affecting a student's designated folder (`student-x/**`) triggers that student's individual workflow file (`student-1.yml` to

	`student-5.yml`). Each workflow independently checks out the code, sets up the required runtime (e.g. Python), installs dependencies, validates syntax, and builds Docker images for that student's frontend, backend/API, and database microservices.
3. Build validation	Each workflow confirms that its microservices build successfully as Docker images and that the code is free of syntax errors before allowing changes to be merged into the main branch. This acts as a quality gate, catching integration-breaking issues early and per-student rather than at the end of the project.
4. Docker Compose build and orchestration	Once individual microservices are validated, the shared `docker-compose.yml` file builds and starts all five students' containers together as a single multi-container application, wiring up networking between each frontend, backend, and database set.
5. Integrated microservices application	All five frontend, backend/API, and database microservice sets run together, communicating via REST APIs, and are linked through the unified home page (`index.html`) with a shared CSS theme.
6. AI-Mode integration	Each backend/API microservice connects to the shared AI-Mode service (Ollama runtime with an approved open-source LLM — Llama, Qwen, or DeepSeek), enabling AI-assisted functionality across all microservices as part of the Plan → Act → Observe → Adapt workflow.
7. Local execution and validation	The integrated application is run and manually tested locally via `docker compose up`, with evidence (logs, screenshots) captured to demonstrate successful startup, container health, and functional CRUD/AI operations.

8.0 Agentic AI Workflow

8.1 Plan → Act → Observe → Adapt diagram



8.2 Workflow explanation and role of AI agents

Stage	Explanation
Plan	When the user submits a request (natural language query or CRUD action) through any frontend microservice, the backend/API microservice will interpret the request and determine the best course of action, whether to connect to the database service and make a direct query or to call upon the AI-mode service.
Act	The backend/API executes the planned action: sending the request to the shared AI-Mode service (Ollama running an approved LLM), and/or performing the relevant database operation (Create, Read, Update, Delete).
Observe	The system evaluates the result returned from the LLM or database - checking whether the response is complete, relevant, and correctly formatted before it is passed back to the user.
Adapt	The result is formatted for display in the frontend. If the response is incomplete, ambiguous, or an error occurred, the system adjusts for example, retrying the query, narrowing the AI prompt, or returning a clarifying message to the user. The cycle then restarts with the next user interaction.

8.3 MCP Server

The project uses a shared Model Context Protocol (MCP) server to make feature tools available through a common interface. The server runs as a local Python process on the host machine, separately from Docker Compose. Feature backends connect to it using the host address and MCP endpoint configured for their environment.

The shared server registers tools contributed by different project features. Depending on the feature, tools can retrieve or operate on information such as student, staff, course, timetable or assessment data. Each tool defines its inputs and returns a structured result. In an AI-assisted workflow, the backend can use the model's request to determine whether a tool is needed, call it through an MCP client, and use the result when forming the response. Some workflows may also call a tool directly from a backend route.

Request flow:

User → feature frontend → feature backend → MCP client → shared MCP server → feature tool/data source → structured result → backend response

8.4 RAG Server

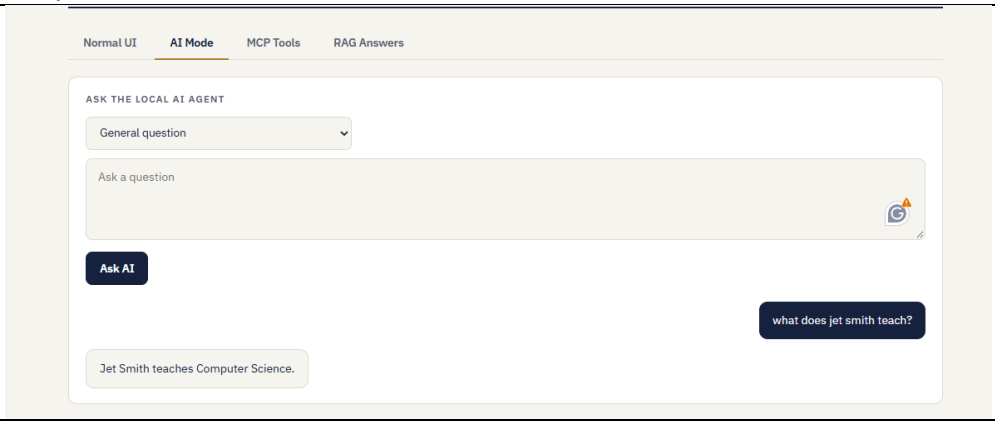
The project uses a shared Retrieval-Augmented Generation (RAG) service to support responses grounded in project data or documents. Like the MCP server, it runs separately from Docker Compose, and feature backends send requests to it over the configured host address.

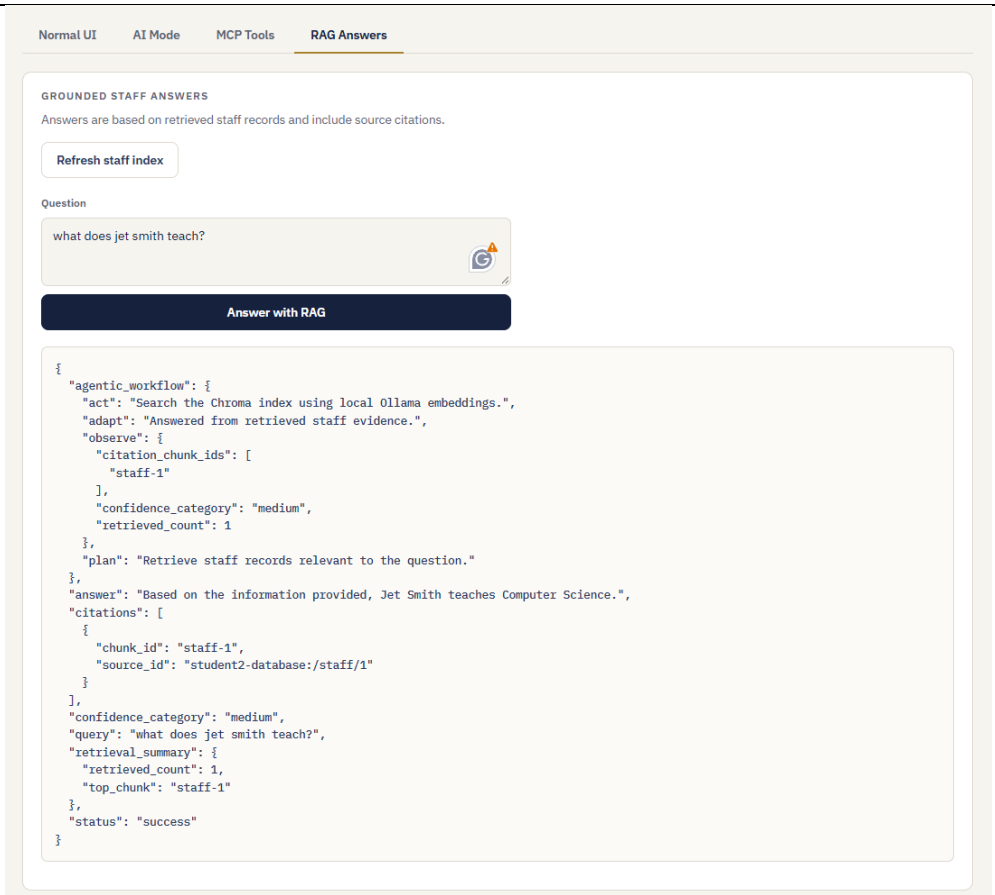
The RAG service routes requests to the appropriate feature domain. Each domain supplies its own knowledge sources and retrieval and answer logic. Depending on the domain, the service can prepare or refresh an index, retrieve information relevant to a query, and return supporting evidence for an answer. Where a domain uses vector retrieval, documents or records are converted into text chunks, embedded, and searched for relevant matches. The service can then use the retrieved evidence to generate a response, include citations, or abstain when the evidence is insufficient.

Request flow:

User question → feature frontend → feature backend → shared RAG service → domain-specific retrieval → evidence check → grounded response with supporting sources or abstention

8.5 Grounded AI Response Flow

Run	Response
Without retrieved evidence	

With retrieved evidvence	

9.0 GitHub Actions Workflows

9.1 Description of student-1.yml - Rigel Rivamonte

The purpose of this workflow is to automate the validation and Docker image builds for the Student Records feature in Student 1, including its MCP Mode and RAG Mode integrations with the shared AI services. It is designed to run only when relevant Student 1 files, the shared MCP/RAG services, the Docker Compose configuration, or the workflow itself are changed, helping to confirm that the feature is functioning correctly before it is merged into the shared project.

Triggers:

- Push: runs on pushes to the rigel or main branches when files inside student-1/, ai-services/mcp-server/, ai-services/rag-server/, docker-compose.yml, or the workflow file are changed

- Pull request: runs when a pull request targets main and the changed files are within student-1/, ai-services/mcp-server/, ai-services/rag-server/, docker-compose.yml, or the workflow file

The workflow performs the following steps:

- Checks out the repository
- Sets up Python 3.11
- Installs the Student 1 enrolment service dependencies, the shared MCP server dependencies, and the shared RAG server dependencies
- Runs the Student Records unit tests
- Validates Python syntax for the database application, database initialisation, Flask application, normal UI routes, AI routes, MCP routes, RAG routes, database API service, LLM client, prompt loader, MCP API service, RAG API service, and HTML formatters
- Validates Python syntax for the Student 1 MCP tool definitions and the shared MCP server
- Validates Python syntax for the Student 1 RAG integration and the shared RAG HTTP server
- Runs the shared MCP server and RAG server unit test suites
- Builds the frontend Docker image
- Builds the enrolment Docker image
- Builds the database Docker image
- Builds the shared MCP service Docker image
- Builds the shared RAG service Docker image
- Validates the Docker Compose configuration
- Verifies that the Docker Compose configuration includes the MCP_ENABLED, MCP_SERVER_URL, RAG_ENABLED, and RAG_SERVER_URL settings required for Release 1

When all checks pass, this workflow confirms that the Student 1 feature, along with its MCP Mode and RAG Mode functionality, is functionally valid, syntactically correct, and ready to be containerised before it is integrated into the wider application.

9.2 Description of student-2.yml - Jeriko Arceo

The purpose of this workflow is to build and validate the Student 2 staff management feature and its Release 1 MCP and RAG integrations. It runs when relevant project files change.

Triggers:

- Push to the jeriko or jeriko-workingAttempt branches when changes affect student-2/, ai-services/mcp-server/, ai-services/rag-server/, or the root docker-compose.yml.
- Pull request targeting main or jeriko-workingAttempt when changes affect those same paths.

The workflow performs these steps:

- Checks out the repository.
- Sets up Python 3.11 and installs the Student 2 backend and shared MCP dependencies.
- Validates the syntax of selected Student 2, MCP and RAG Python files.
- Runs the MCP, RAG and Student 2 backend behaviour test suites.
- Builds Docker images for the Student 2 frontend, backend and database, as well as the MCP and RAG services.
- Validates the root Docker Compose configuration with docker compose config.

A successful run provides automated checks that the selected Python files compile, the Release 1 behaviour tests pass, the listed Docker images build, and the shared Compose configuration resolves correctly. The workflow builds MCP and RAG images for CI validation; in the documented Release 1 setup, the shared MCP and RAG servers are started as local host processes rather than Compose services.

9.3 Description of student-3.yml

The purpose of student-3.yml is to automate the build and validation of the Timetable & Class Scheduling feature, triggering only on relevant code changes.

Release 1 update. The workflow keeps MCP and RAG integrated but disabled during CI:

- Triggers also include the Timetable files in the shared AI services (timetable_tools.py, timetable_rag.py, timetable_policy.md and their tests)
- Runs the backend MCP/RAG route tests, Timetable MCP tool tests and Timetable RAG tests, with the MCP server, RAG server and Ollama mocked
- Starts the backend with MCP_ENABLED=false and RAG_ENABLED=false and asserts that /mcp/call and /rag/answer return 503 “disabled”
- Checks the new /api/timetable endpoint, then builds the backend and frontend Docker images

Successful run: https://github.com/rigels51/ASD-Project-Group18/actions/runs/36728799288

9.4 Description of student-4.yml - Yi Zhang

Student 4 uses the YiZhang.yml workflow to validate the Course and Enrollment Management feature for Release 1. The workflow runs when changes are pushed to the Yi-Zhang branch or when a related pull request is created. YiZhang

The workflow:

- Validates the Student 4 database and backend Python files.
- Validates the MCP and RAG integration files.
- Builds the frontend, enrolment service, and database Docker images.
- Validates the Docker Compose configuration.
- Checks that the MCP and RAG configuration is included in docker-compose.yml.

The workflow helps detect Python, Docker, MCP, and RAG configuration problems before integration.

9.5 Description of student-5.yml

The Student 5 GitHub Actions workflow is used to automatically validate the Assessment & Grades Management feature when changes are pushed or submitted through a pull request affecting the Student 5 files.

The workflow uses Python 3.11 and performs the following steps:

- Checks out the repository.
- Sets up Python 3.11.
- Installs the required Python dependencies and testing packages.
- Builds the Assessment Database Service Docker image.
- Builds the Assessment Service Docker image.
- Builds the Assessment Frontend Service Docker image.
- Creates the required Docker network.
- Starts the database and backend services.
- Performs smoke tests against the database and Assessment API.
- Runs the automated Student 5 test suite.
- Cleans up the test containers after validation.

The main purpose of the workflow is to identify Python, API, testing, and Docker problems before the Assessment & Grades Management feature is integrated into the main application.

For Release 1, the Student 5 workflow was extended to validate the new MCP and RAG integrations.

The workflow continues to build and test the Assessment & Grades frontend, backend, and database services. It also runs the Student 5 automated tests and checks the MCP and RAG backend routes with the integrations disabled.

The CI environment uses MCP_ENABLED=false and RAG_ENABLED=false so that the workflow does not depend on the shared MCP and RAG servers being available. The workflow verifies that the corresponding routes correctly report that the integrations are disabled.

This allows the existing Assessment & Grades functionality to continue being tested independently while also validating the Release 1 configuration.

10.0 Implementation Summary

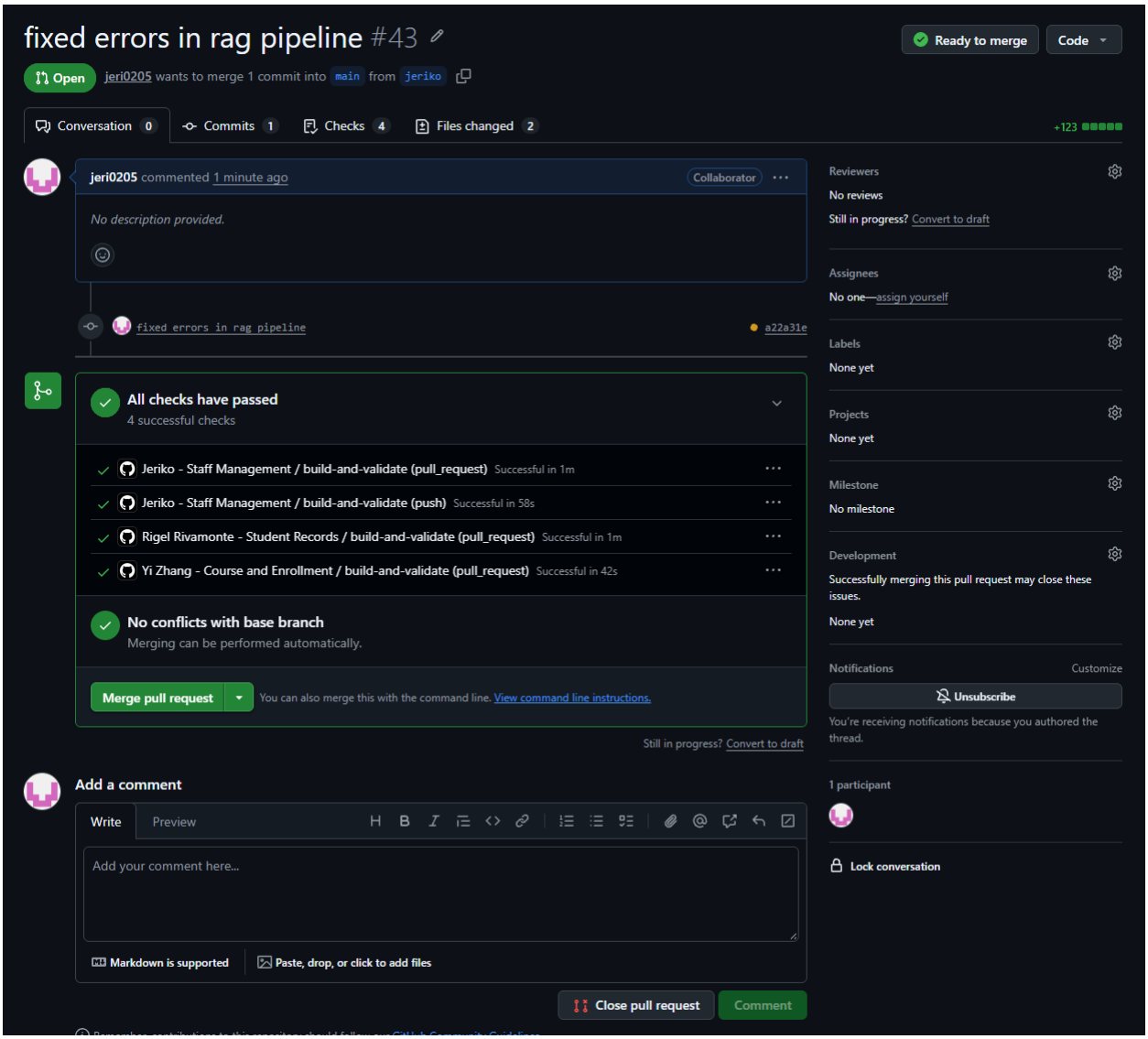
10.1 Release 1 scope and completed features

Release 1 focused on extending the Release 0 microservice architecture with shared MCP and RAG capabilities. The major completed features were:

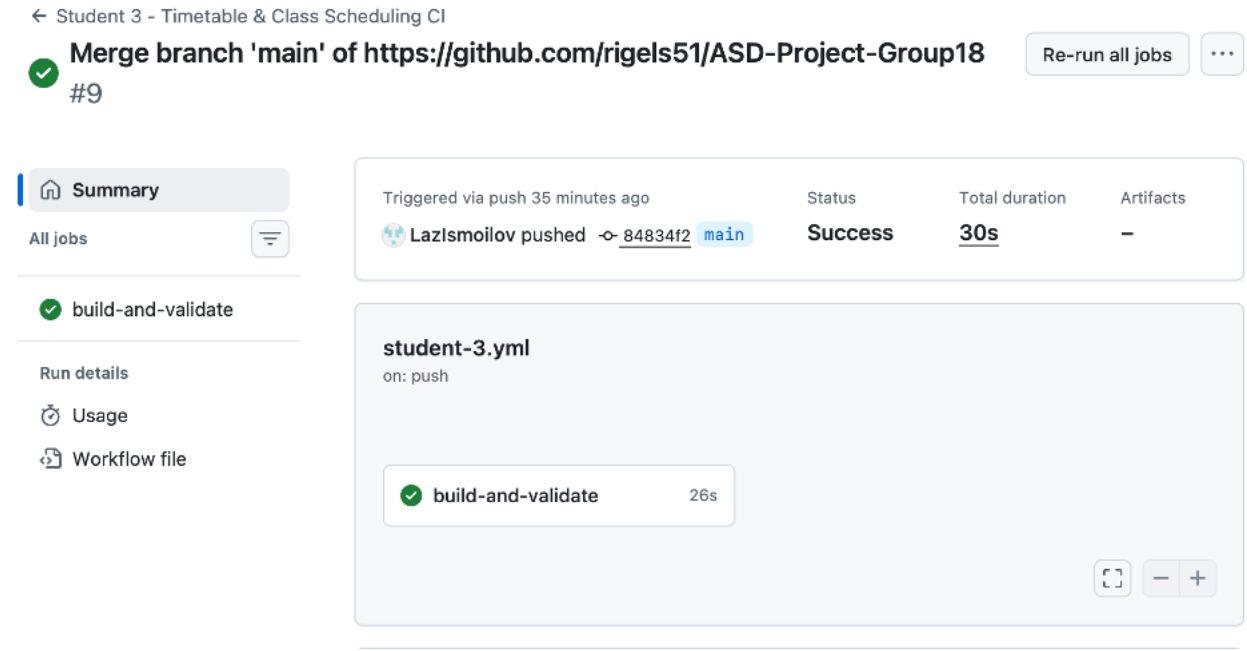
- Shared MCP server integrated with the individual feature backends.
- Predefined MCP tools providing controlled access to supported application data.
- Shared RAG server providing retrieval-grounded context for natural-language questions.
- RAG Mode answers grounded in retrieved application data rather than relying solely on model-generated information, alongside the existing AI Mode.
- Updated Docker Compose architecture supporting the additional shared AI services.
- Architecture and workflow documentation updated to describe the MCP/RAG request and retrieval paths.
- Evidence collection established for MCP requests, RAG retrieval, and grounded responses, to be completed by running the stack locally.

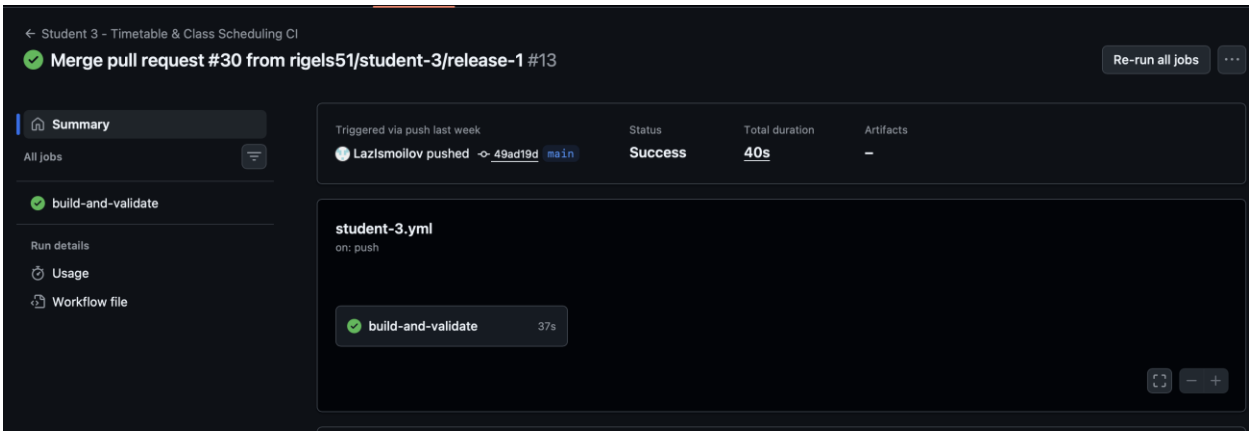
11.0 GitHub Actions Evidence (i)

Jeriko Arceo

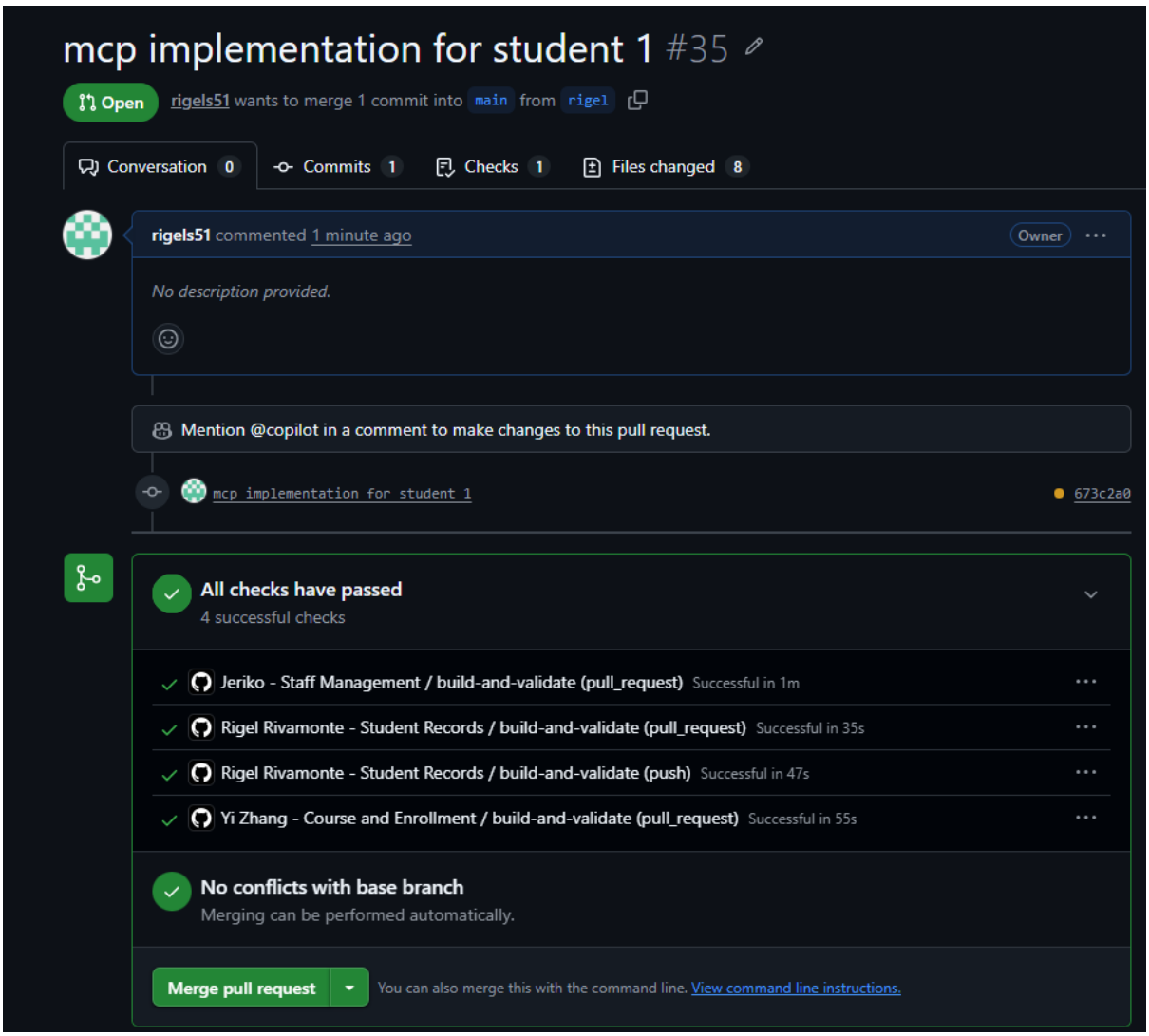


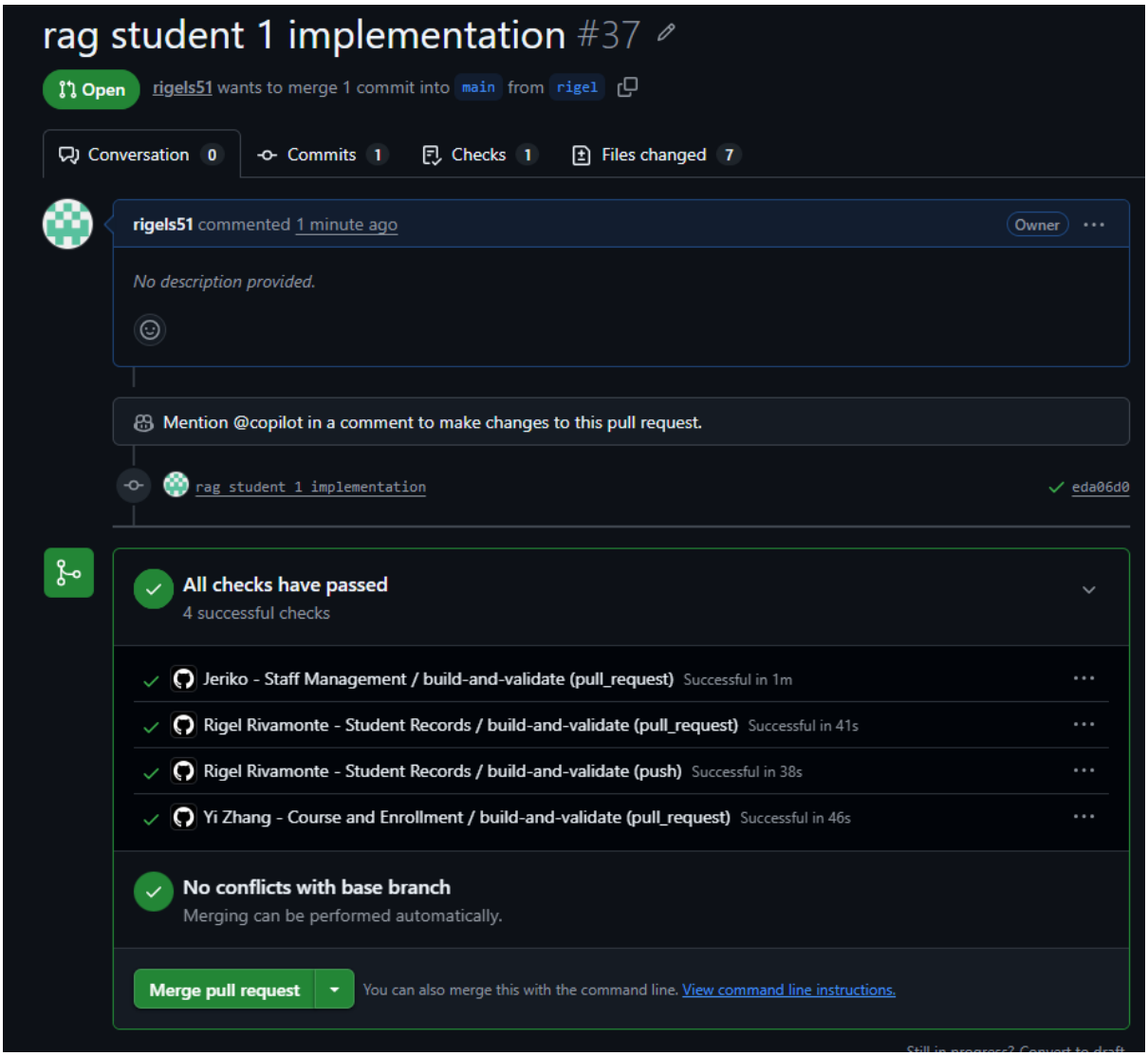
Lazizbek Ismoilov

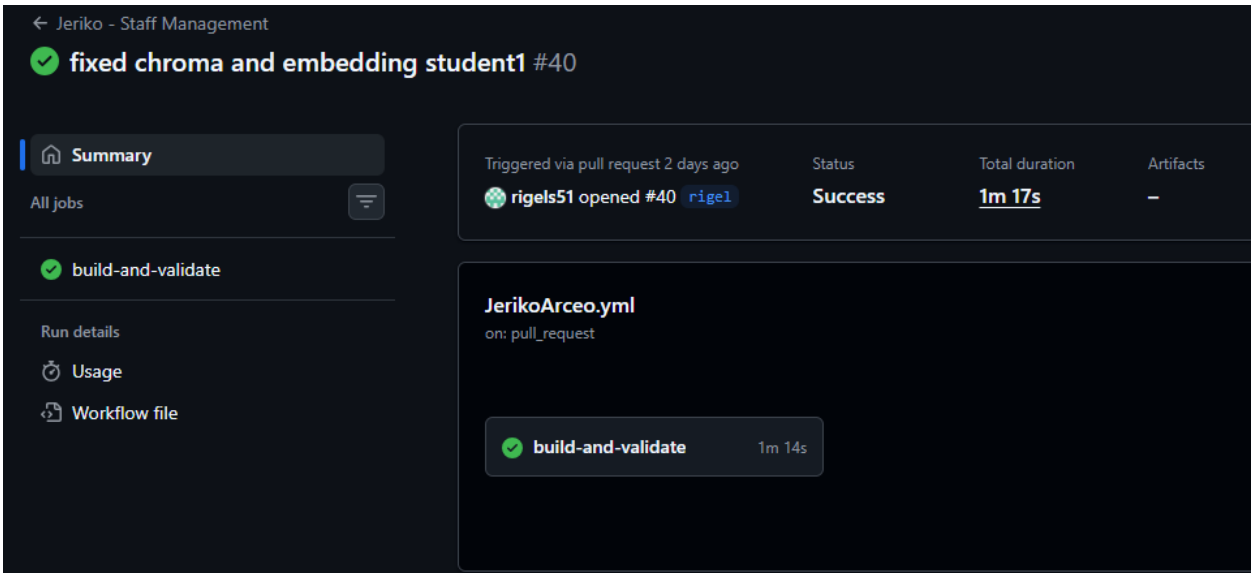

Student 3 - Timetable & Class Scheduling CI
Merge branch 'main' of https://github.com/rigels51/ASD-Project-Group18 #9
Re-run all jobs
Summary
All jobs
build-and-validate
Run details
Usage
Workflow file
Triggered via push 35 minutes ago
LazIsmoilov pushed 84834f2 main
Status
Success
Total duration
30s
Artifacts
–
student-3.yml
on: push
build-and-validate 26s


Student 3 - Timetable & Class Scheduling CI
Merge pull request #30 from rigels51/student-3/release-1 #13
Re-run all jobs
Summary
All jobs
build-and-validate
Run details
Usage
Workflow file
Triggered via push last week
LazIsmoilov pushed 49ad19d main
Status
Success
Total duration
40s
Artifacts
–
student-3.yml
on: push
build-and-validate 37s

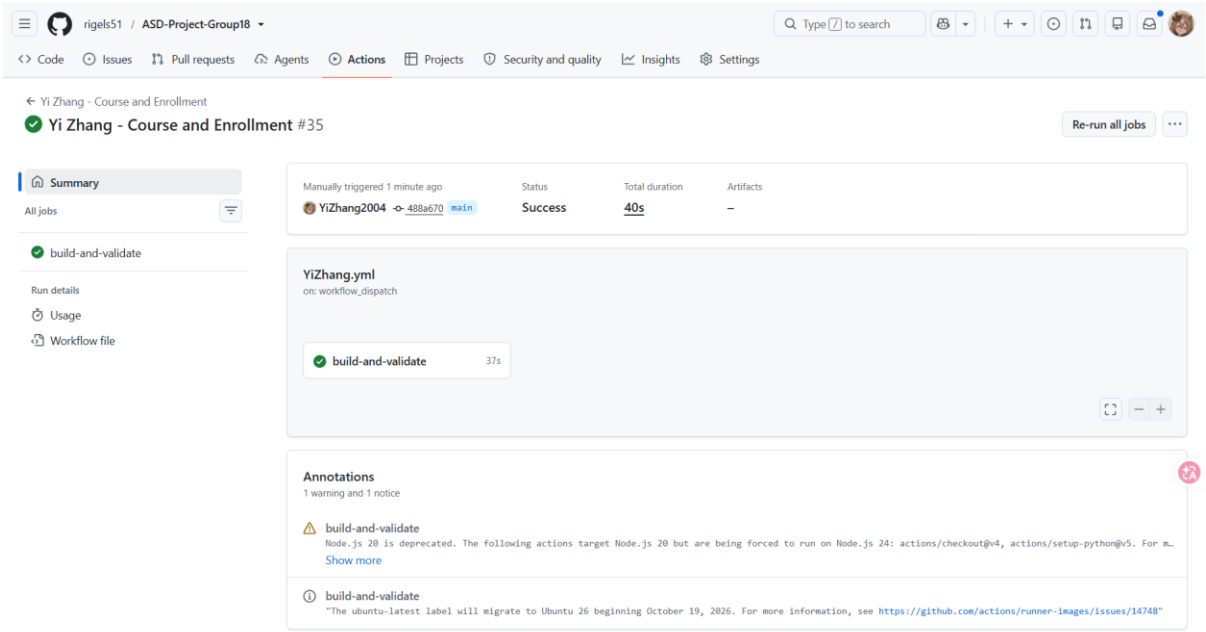
Rigel Rivamonte



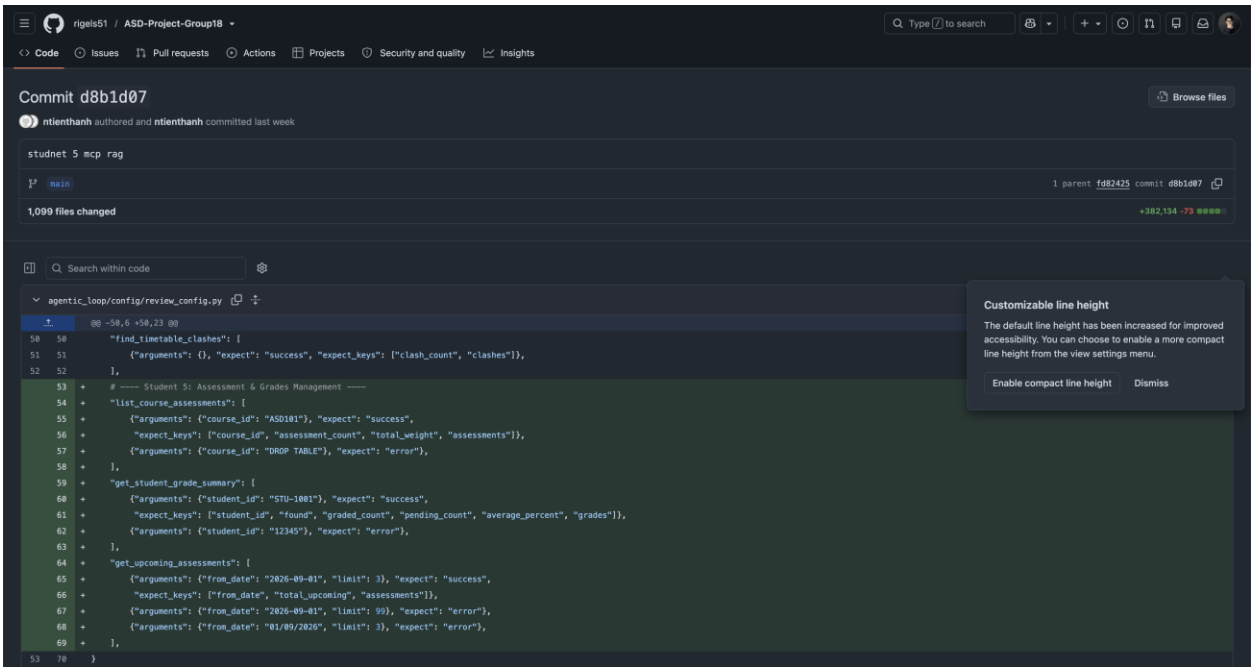

rag student 1 implementation #37
Open
rigels51 wants to merge 1 commit into main from rigel
Conversation 0
Commits 1
Checks 1
Files changed 7
rigels51 commented 1 minute ago
Owner
No description provided.
Mention @copilot in a comment to make changes to this pull request.
rag student 1 implementation
eda06d0
All checks have passed
4 successful checks
Jeriko - Staff Management / build-and-validate (pull_request) Successful in 1m
Rigel Rivamonte - Student Records / build-and-validate (pull_request) Successful in 41s
Rigel Rivamonte - Student Records / build-and-validate (push) Successful in 38s
Yi Zhang - Course and Enrollment / build-and-validate (pull_request) Successful in 46s
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line. View command line instructions.



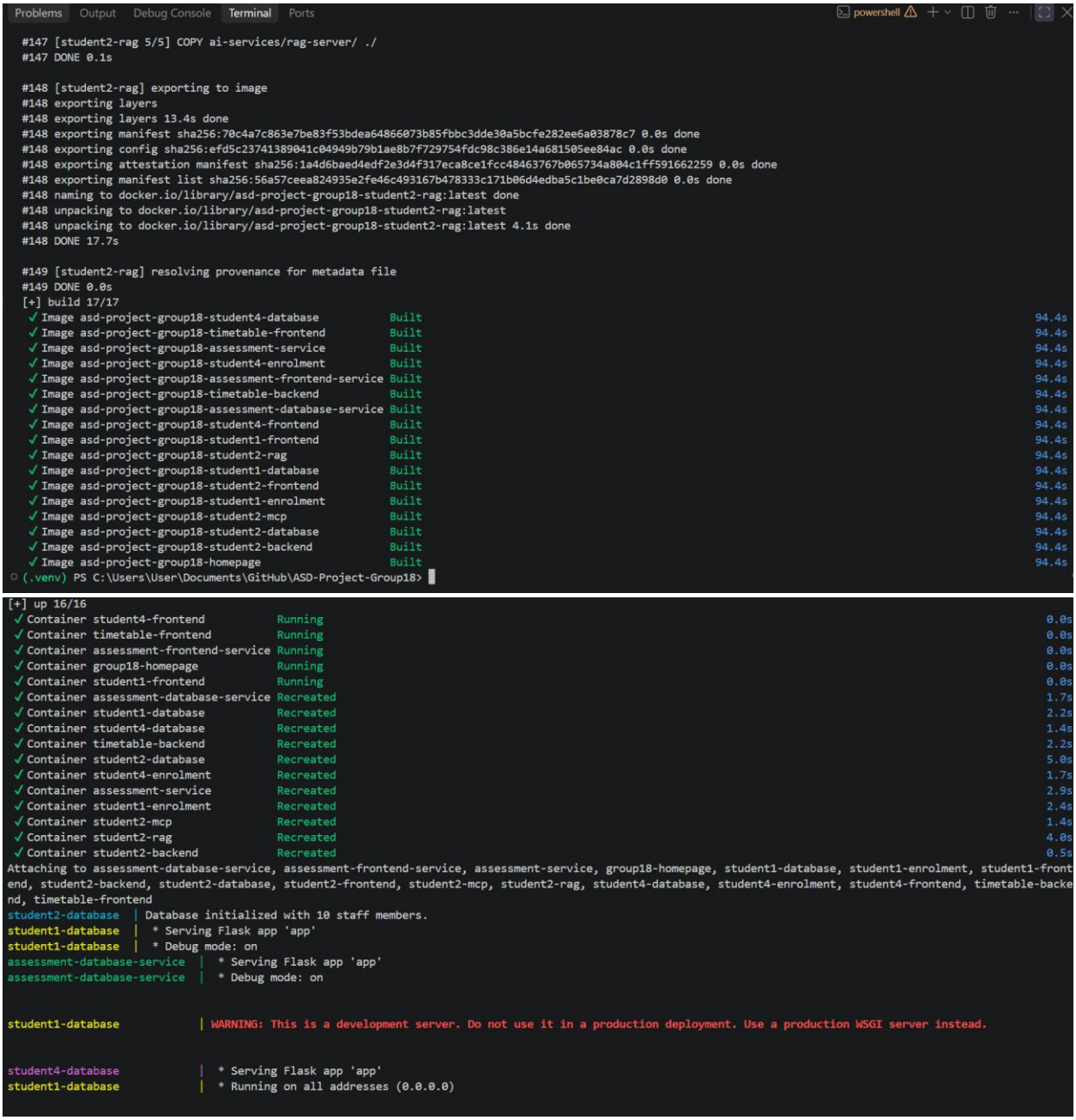
Yi Zhang



Vu Tien Thanh Nguyen


rigels51 / ASD-Project-Group18
Type / to search
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Commit d8b1d07
Browse files
ntienthanh authored and ntienthanh committed last week
studnet 5 mcp rag
main
1 parent fd82425 commit d8b1d07
1,099 files changed
+382,134 -73
Search within code
agentic_loop/config/review_config.py
@@ -50,6 +50,23 @@
"find_timetable_clashes": [
{"arguments": {}, "expect": "success", "expect_keys": ["clash_count", "clashes"]},
],
+ # ---- Student 5: Assessment & Grades Management ----
+ "list_course_assessments": [
+ {"arguments": {"course_id": "ASD101"}, "expect": "success",
+ "expect_keys": ["course_id", "assessment_count", "total_weight", "assessments"]},
+ {"arguments": {"course_id": "DROP TABLE"}, "expect": "error"},
+],
+ "get_student_grade_summary": [
+ {"arguments": {"student_id": "STU-1001"}, "expect": "success",
+ "expect_keys": ["student_id", "found", "graded_count", "pending_count", "average_percent", "grades"]},
+ {"arguments": {"student_id": "12345"}, "expect": "error"},
+],
+ "get_upcoming_assessments": [
+ {"arguments": {"from_date": "2026-09-01", "limit": 3}, "expect": "success",
+ "expect_keys": ["from_date", "total_upcoming", "assessments"]},
+ {"arguments": {"from_date": "2026-09-01", "limit": 99}, "expect": "error"},
+ {"arguments": {"from_date": "01/09/2026", "limit": 3}, "expect": "error"},
+],
}
Customizable line height
The default line height has been increased for improved accessibility. You can choose to enable a more compact line height from the view settings menu.
Enable compact line height
Dismiss

12.0 Docker Compose Evidence



```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

 #147 [student2-rag 5/5] COPY ai-services/rag-server/ ./
 #147 DONE 0.1s

 #148 [student2-rag] exporting to image
 #148 exporting layers
 #148 exporting layers 13.4s done
 #148 exporting manifest sha256:70c4a7c863e7be83f53bdea64866073b85fbbc3dde30a5bcfe282ee6a03878c7 0.0s done
 #148 exporting config sha256:efd5c23741389041c04949b79b1ae8b7f729754fdc98c386e14a681505ee84ac 0.0s done
 #148 exporting attestation manifest sha256:1a4d6baed4edf2e3d4f317eca8ce1fcc48463767b065734a804c1ff591662259 0.0s done
 #148 exporting manifest list sha256:56a57ceea824935e2fe46c493167b478333c171b06d4edba5c1be0ca7d2898d0 0.0s done
 #148 naming to docker.io/library/asd-project-group18-student2-rag:latest done
 #148 unpacking to docker.io/library/asd-project-group18-student2-rag:latest
 #148 unpacking to docker.io/library/asd-project-group18-student2-rag:latest 4.1s done
 #148 DONE 17.7s

 #149 [student2-rag] resolving provenance for metadata file
 #149 DONE 0.0s
 [+] build 17/17
  ✓ Image asd-project-group18-student4-database          Built                                94.4s
  ✓ Image asd-project-group18-timetable-frontend         Built                                94.4s
  ✓ Image asd-project-group18-assessment-service         Built                                94.4s
  ✓ Image asd-project-group18-student4-enrolment         Built                                94.4s
  ✓ Image asd-project-group18-assessment-frontend-service Built                               94.4s
  ✓ Image asd-project-group18-timetable-backend          Built                                94.4s
  ✓ Image asd-project-group18-assessment-database-service Built                               94.4s
  ✓ Image asd-project-group18-student4-frontend          Built                                94.4s
  ✓ Image asd-project-group18-student1-frontend          Built                                94.4s
  ✓ Image asd-project-group18-student2-rag               Built                                94.4s
  ✓ Image asd-project-group18-student1-database          Built                                94.4s
  ✓ Image asd-project-group18-student2-frontend          Built                                94.4s
  ✓ Image asd-project-group18-student1-enrolment         Built                                94.4s
  ✓ Image asd-project-group18-student2-mcp               Built                                94.4s
  ✓ Image asd-project-group18-student2-database          Built                                94.4s
  ✓ Image asd-project-group18-student2-backend           Built                                94.4s
  ✓ Image asd-project-group18-homepage                   Built                                94.4s
○ (.venv) PS C:\Users\User\Documents\GitHub\ASD-Project-Group18>
```

```
[+] up 16/16
 ✓ Container student4-frontend           Running                                             0.0s
 ✓ Container timetable-frontend          Running                                             0.0s
 ✓ Container assessment-frontend-service Running                                             0.0s
 ✓ Container group18-homepage            Running                                             0.0s
 ✓ Container student1-frontend           Running                                             0.0s
 ✓ Container assessment-database-service Recreated                                           1.7s
 ✓ Container student1-database           Recreated                                           2.2s
 ✓ Container student4-database           Recreated                                           1.4s
 ✓ Container timetable-backend           Recreated                                           2.2s
 ✓ Container student2-database           Recreated                                           5.0s
 ✓ Container student4-enrolment          Recreated                                           1.7s
 ✓ Container assessment-service          Recreated                                           2.9s
 ✓ Container student1-enrolment          Recreated                                           2.4s
 ✓ Container student2-mcp                Recreated                                           1.4s
 ✓ Container student2-rag                Recreated                                           4.0s
 ✓ Container student2-backend            Recreated                                           0.5s
Attaching to assessment-database-service, assessment-frontend-service, assessment-service, group18-homepage, student1-database, student1-enrolment, student1-front
end, student2-backend, student2-database, student2-frontend, student2-mcp, student2-rag, student4-database, student4-enrolment, student4-frontend, timetable-backe
nd, timetable-frontend
student2-database  | Database initialized with 10 staff members.
student1-database  |  * Serving Flask app 'app'
student1-database  |  * Debug mode: on
assessment-database-service  |  * Serving Flask app 'app'
assessment-database-service  |  * Debug mode: on


student1-database           | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.


student4-database           |  * Serving Flask app 'app'
student1-database           |  * Running on all addresses (0.0.0.0)
```

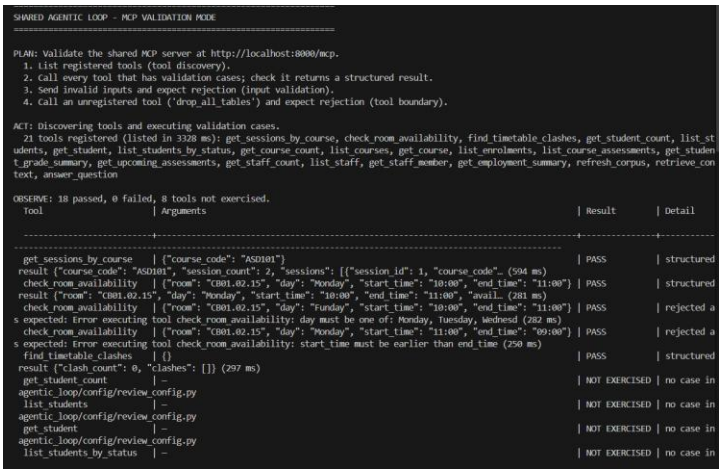
13.0 Agentic Loop Workflow Record

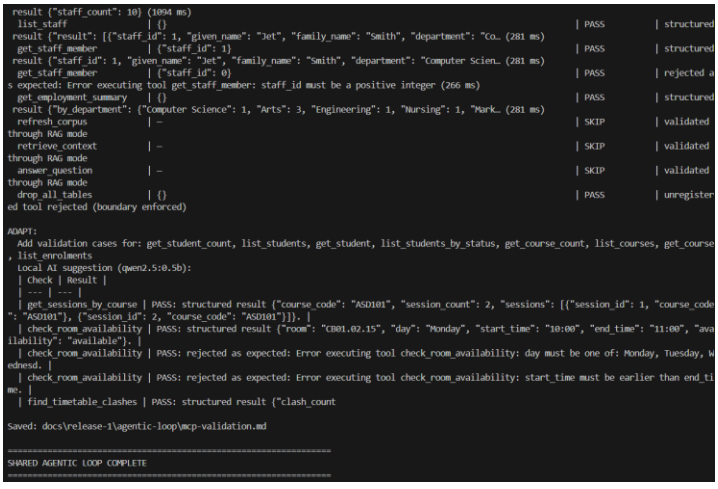
13.1 Shared Agentic Loop Development Contributions

A shared Agentic Loop was added at the project level using agentic_loop/main.py. It provides a common entry point for running feature-level Agentic Loops.

The shared loop currently runs the Course and Enrollment Management Agentic Loop. It checks database records, API endpoints, AI responses, and review results.

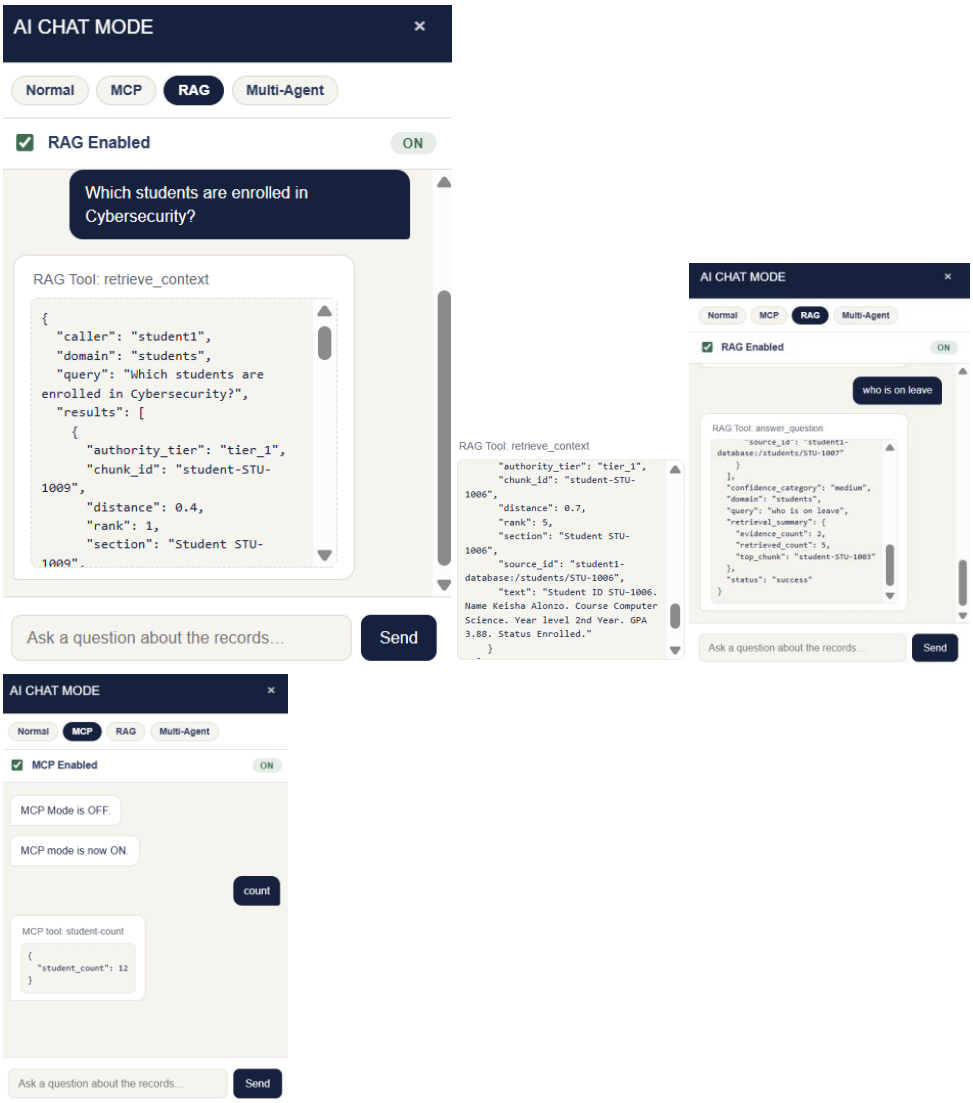
The workflow follows the Plan → Act → Observe → Review → Human Decision → Adapt process and completes with a final validation result.





13.2 Individual Prompt Assets (i)

Rigel Rivamonte



The Student Records RAG Mode uses the shared RAG server with the local Ollama models qwen2.5:0.5b (chat) and nomic-embed-text (embeddings). Before answering, student records are fetched from the Student 1 database and converted into per-student text chunks plus one aggregate "student summary" chunk, each embedded and stored in a persistent ChromaDB collection. When a question is asked, the query is embedded, the most relevant chunks are retrieved, and only chunks within a set relevance distance are passed to the model as evidence. The generation prompt instructs the model to:

- Answer using only the retrieved evidence.
- Not use outside knowledge.

- Not guess or invent any information.
- Ensure every factual statement comes from the evidence.
- Cite the relevant chunk ID for each statement.
- Keep the answer short.
- Reply with a fixed "insufficient context" message when the evidence does not support an answer.

Example supported questions include:

- "How many students are on record?"
- "What course is a given student enrolled in?"
- A question with no matching evidence (triggers the fixed insufficient-context response rather than a guess).

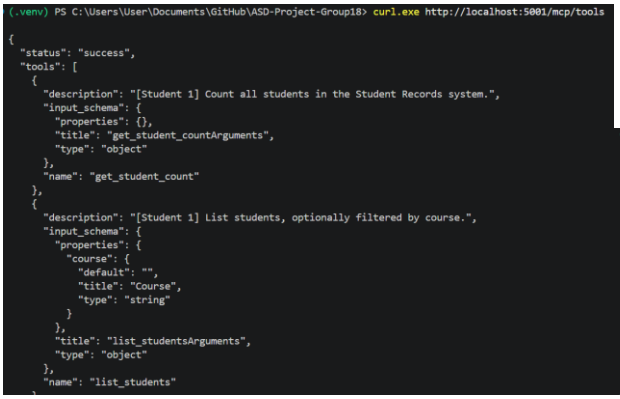
The Student Records MCP Mode exposes read-only tools on the shared MCP server, registered alongside the other features' tools under the server-wide instruction to use the registered tools for factual lookups and not infer missing information. Rather than a natural-language prompt, each Student 1 tool carries its own name, description, and schema, which instructs any calling agent to:

- Use get_student_count for an exact total of students on record.
- Use list_students, optionally filtered by course, to list matching students.
- Use get_student with an exact student ID (e.g. STU-1001) to retrieve one record.
- Use list_students_by_status, optionally filtered by status (Enrolled, On Leave, Graduated), to list matching students.

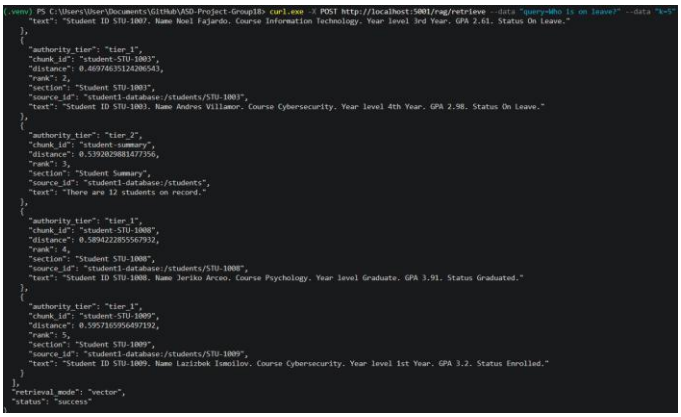
Example supported requests include:

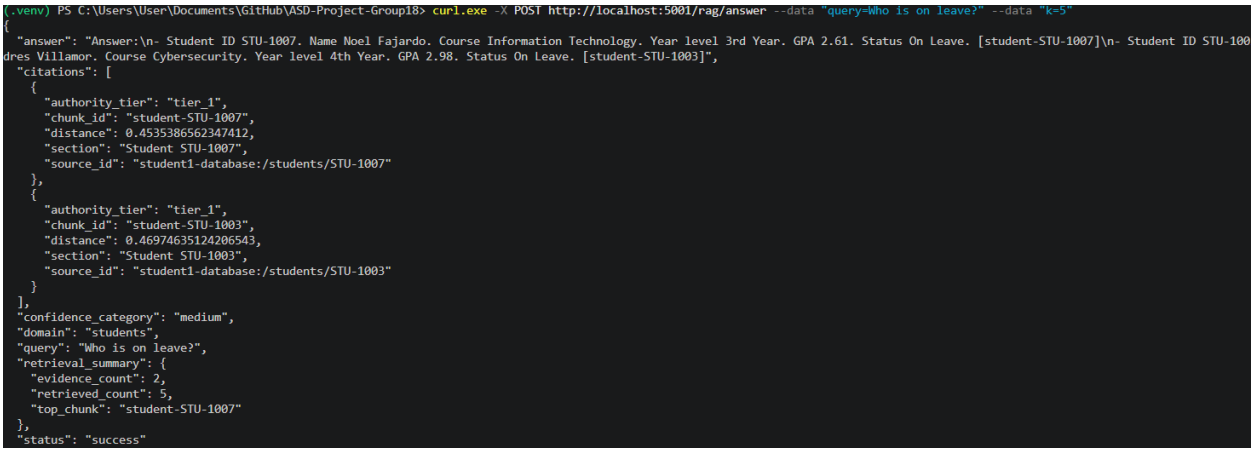
- "How many students are there?" → get_student_count

Local Terminal Validation









Jeriko Arceo

Student 2 Staff Registry RAG Mode

The Staff Registry RAG Mode uses the shared RAG server with the local Ollama models qwen2.5:0.5b for answer generation and nomic-embed-text for embeddings. When the staff index is refreshed, records are fetched from the Student 2 database and converted into one text chunk per staff member. Each chunk contains the staff ID, name, department and employment type; email addresses are excluded. The chunks are embedded and stored in a persistent ChromaDB collection.

When a question is asked, the query is embedded and the most relevant chunks are retrieved—five by default, up to ten. Results beyond the configured relevance-distance threshold are excluded from the answer evidence. The generation prompt instructs the model to:

- Answer using only the supplied staff evidence.
- Cite factual claims with the corresponding staff chunk ID.
- Say when the evidence does not answer the question.
- Abstain when retrieval confidence is low.

Exact staff-count and employment-type questions use a separate path that queries the live staff database rather than relying on vector retrieval.

Example supported questions include:

- “Which staff members work in Arts?”
- “Who is staff member 3?”
- “How many staff members are in the registry?” — answered from the live database.
- A question with no relevant evidence — the service abstains rather than inventing an answer.

Student 2 Staff Registry MCP Mode

The Staff Registry MCP Mode uses the shared MCP server, which registers Student 2’s read-only staff tools alongside tools contributed by other project features. The tools have defined names, descriptions and input schemas. The Student 2 backend exposes an allowlist and validates tool names and arguments before forwarding calls to the MCP server.

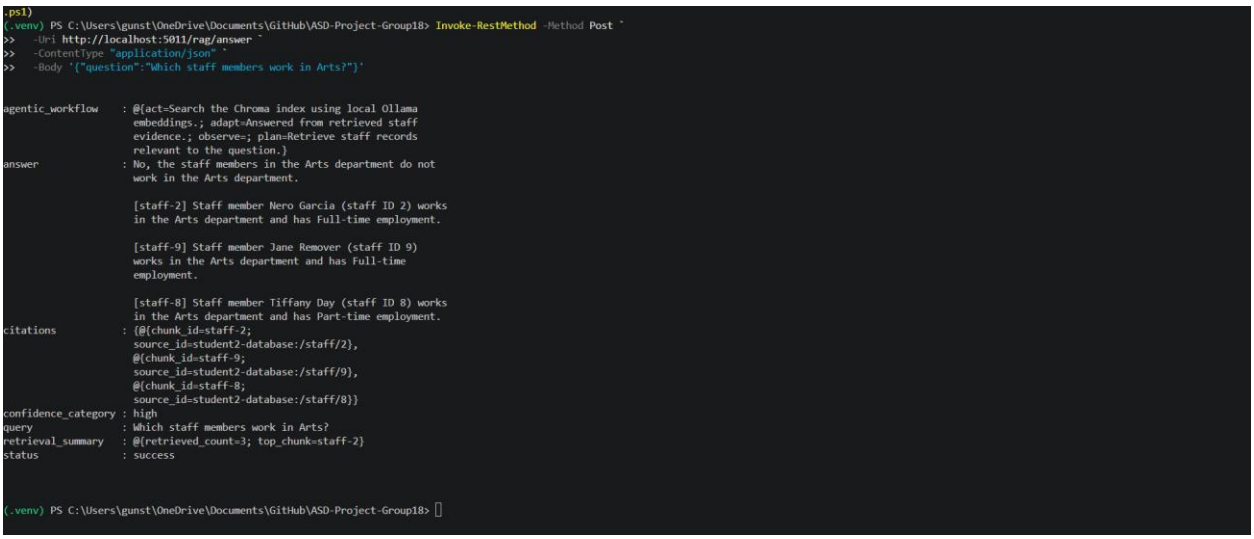
Student 2’s staff tools include:

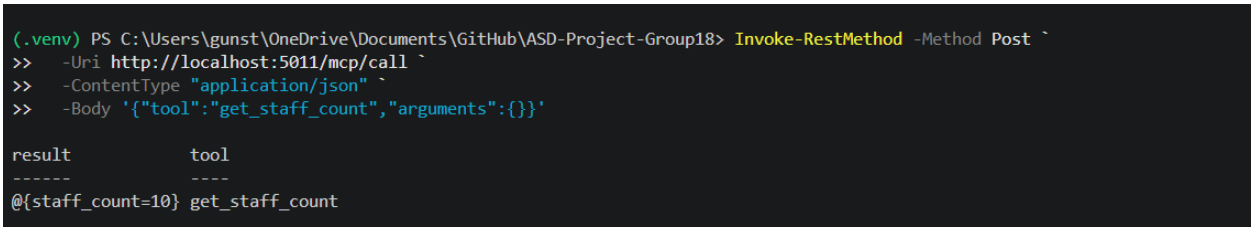
- get_staff_count — returns the number of staff records.
- list_staff — lists staff, optionally filtered by department.
- get_staff_member — retrieves one staff record by ID.
- get_employment_summary — summarizes staff by department and employment type.
- The shared server also exposes RAG-related tools such as refresh_corpus, retrieve_context and answer_question. Staff directory and member results omit email addresses.

Example supported requests include:

- “How many staff members are there?” → get_staff_count
- “List staff in Arts.” → list_staff with a department argument.
- “Get staff member 3.” → get_staff_member with a staff ID.

Local Terminal Validation



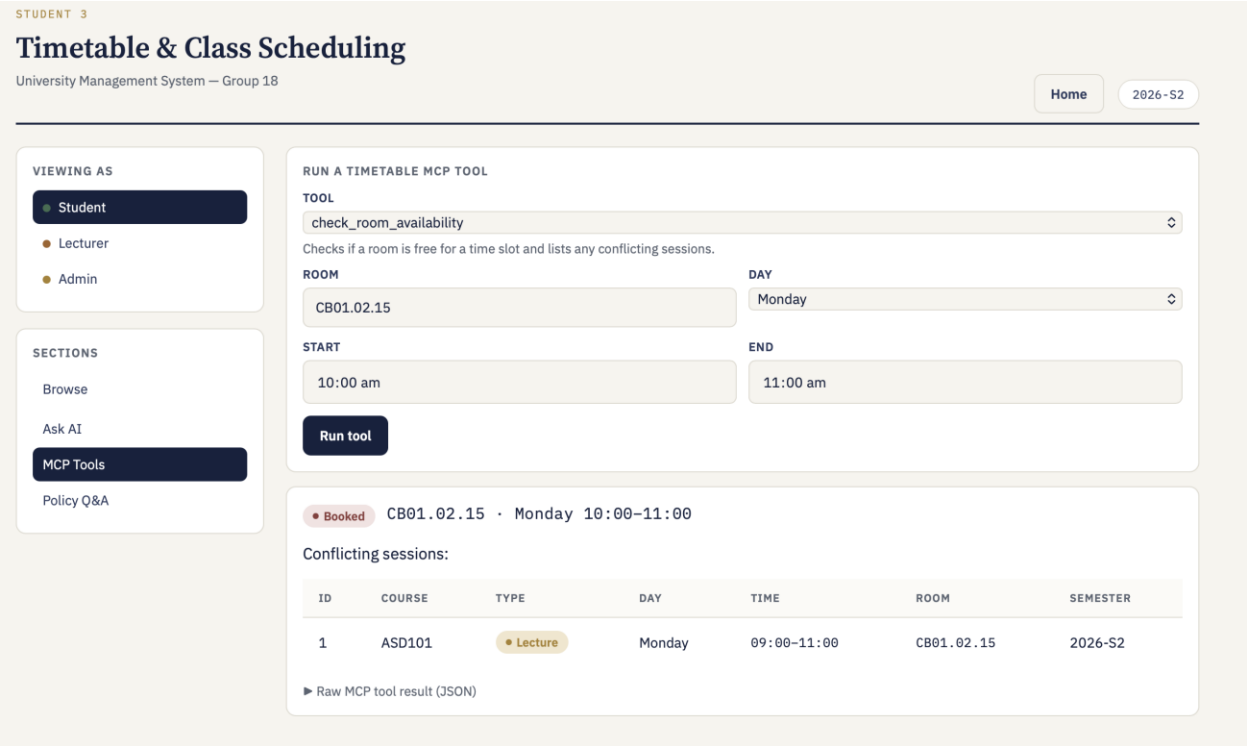


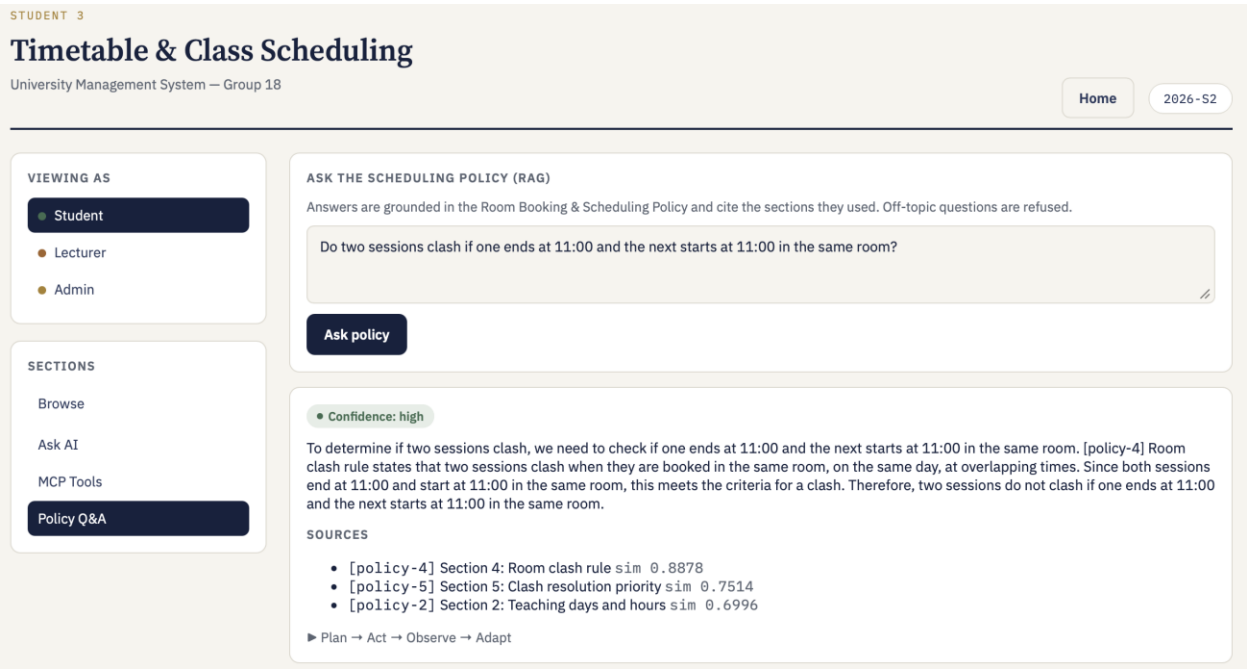
Lazizbek Ismoilov

Release 1 RAG prompt (timetable domain). Only the retrieved policy sections are sent to qwen2.5:0.5b, with these rules: use ONLY the policy evidence below; cite every claim with its exact label, for example [policy-4]; answer in at most three sentences; if the evidence does not answer the question, reply exactly “Insufficient evidence in the scheduling policy to answer this question.” If no section reaches the 0.55 similarity threshold, the prompt is not sent at all.

Example RAG questions:

- Do two sessions clash if one ends at 11:00 and the next starts at 11:00 in the same room? (high confidence, cites policy-4)
- How much notice is required before a session is moved?
- Who won the 2022 World Cup? (insufficient-context response, no citations)





Yi Zhang

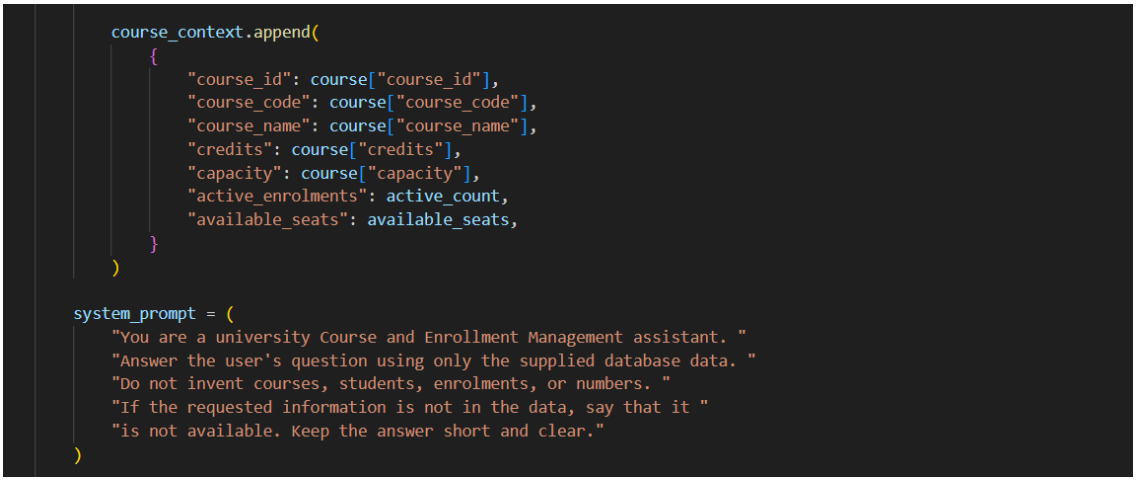
AI prompts are used in the Course and Enrollment Management feature to control AI responses in AI Mode, MCP Mode, and RAG Mode.

For AI Mode, the model provides short answers using current course and enrolment data and avoids inventing information.

For MCP Mode, the system uses structured MCP tools to retrieve course and enrolment information from the application services.

For RAG Mode, the model answers questions using retrieved context from the shared RAG server. The prompt instructs the model to use the provided context, include source citations, and avoid unsupported information. If relevant information cannot be retrieved, the system returns an insufficient-context response.

This prompt design helps keep AI responses clear, relevant, and grounded in the application data.



```
        course_context.append(
            {
                "course_id": course["course_id"],
                "course_code": course["course_code"],
                "course_name": course["course_name"],
                "credits": course["credits"],
                "capacity": course["capacity"],
                "active_enrolments": active_count,
                "available_seats": available_seats,
            }
        )

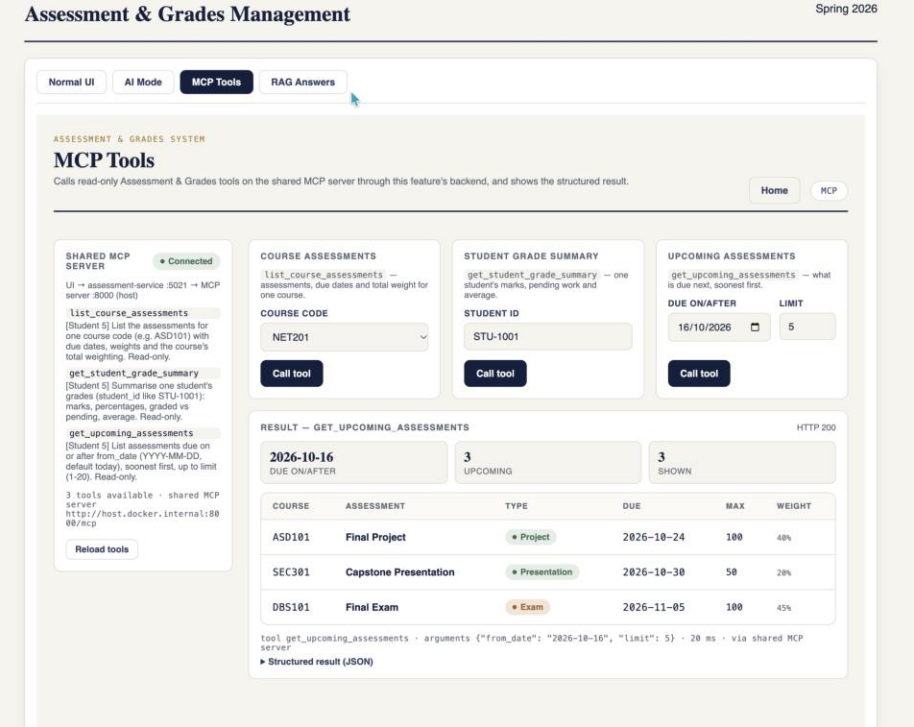
    system_prompt = (
        "You are a university Course and Enrollment Management assistant. "
        "Answer the user's question using only the supplied database data. "
        "Do not invent courses, students, enrolments, or numbers. "
        "If the requested information is not in the data, say that it "
        "is not available. Keep the answer short and clear."
    )
```

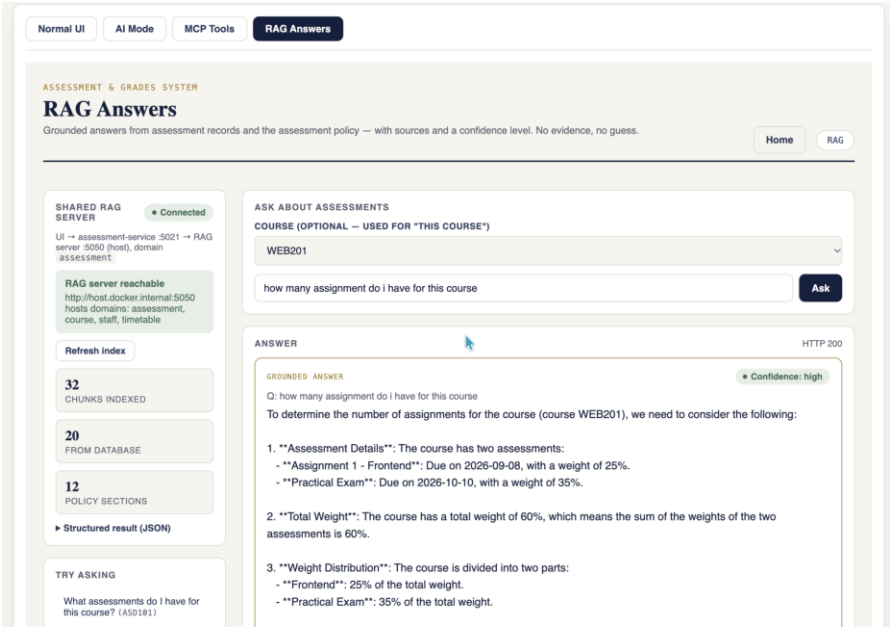
```
348
349     prompt = f"""
350 Answer the question using ONLY the evidence below.
351
352 Strict rules:
353 - Do not use outside knowledge.
354 - Do not guess.
355 - Do not invent any information.
356 - Do not add a university, country, school,
357   department or organisation unless it appears
358   in the evidence.
359 - Every factual statement must come from
360   the evidence.
361 - Cite the relevant chunk ID.
362 - Keep the answer short.
363 - If the evidence is insufficient, answer exactly:
364 {INSUFFICIENT}
365
366 Question:
367 {query}
368
369 Evidence:
370 {context}
371 """
```

Vu Tien Thanh Nguyen

The following screenshots provide evidence of the Release 1 MCP and RAG functionality implemented for the Assessment & Grades Management feature.

- **MCP Tools tab:** Shows the Assessment & Grades MCP interface, including the available Student 5 tools and their inputs and results.
- **Course Assessments:** Demonstrates the list_course_assessments tool retrieving assessment information for a selected course.
- **Student Grade Summary:** Demonstrates the get_student_grade_summary tool and its input validation when an invalid student ID is provided.
- **RAG Answers tab:** Shows the Assessment & Grades RAG interface for asking natural-language questions about assessment information.
- **Grounded RAG Response:** Demonstrates a supported course-related question returning an answer with sources and confidence information.





13.3 Review Records

Student 4 – Course and Enrollment Management

- Reviewed Course and Enrolment CRUD operations.
- Checked Docker and GitHub Actions configuration.

- Verified AI Mode using current course and enrolment data.
- Verified MCP Mode and MCP tool integration.
- Verified RAG retrieval and grounded responses.
- Checked RAG citations and insufficient-context handling.

Student 1 – Student Records Management

- Checked that all student records can be retrieved and displayed
- Verified the search by student name, ID, and course feature
- Tested student creation, update, and deletion functions
- Checked that required fields in the form return appropriate validation errors
- Verified that a missing student record returns a 404 response
- Confirmed if the AI mode uses current student records before answering contextualised questions

Student 5 – Assessment & Grades Management

- Tested list_course_assessments using course ASD101 and confirmed that the expected assessment information was returned.
- Tested an invalid student ID such as 12345 and confirmed that the MCP tool rejected the invalid input.
- Tested MCP tool restrictions to ensure that unsupported tools outside the Student 5 tool set were not accessible.
- Tested a grounded assessment question for ASD101 and confirmed that the response included supporting evidence, citations, and a confidence level.
- Tested a question using assessment policy information and confirmed that relevant policy evidence was retrieved.
- Tested an unknown course such as XYZ999 and confirmed that the system returned insufficient evidence instead of guessing.
- Tested an unrelated question such as "Who won the 2022 football World Cup?" and confirmed that the system returned insufficient evidence.

14.0 Known Issues and Limitations

14.1 Outstanding Issues/Bugs

Student Records Management (Student 1):

- No uniqueness check on email, so duplicate values can be stored.
- AI Mode (/ask, /ask-with-context) depends on the local Ollama service being installed and running

Staff Management (Student 2):

- Some department searches may fail for seeded records.
- Staff changes are saved in the database, but the RAG index is refreshed through a separate operation. Until it is refreshed, retrieval may return outdated records.

Timetable & Class Scheduling (Student 3):

- Lecturer "my sessions" lookup by staff_id is currently a placeholder — staff_id is unpopulated until Staff Management integration confirms a shared ID format
- Clash detection is $O(n^2)$ — fine at current scale (10s of sessions) but would need indexing at real university scale
- Release 1: the shared MCP and RAG servers must be started manually on the host; if they are not running, the MCP Tools and Policy Q&A tabs show "Unavailable" (CRUD and AI-Mode keep working)
- Release 1: qwen2.5:0.5b occasionally words a correct, correctly cited RAG answer awkwardly; the citations let the user verify the source

Course and Enrollment Management (Student 4):

- Student enrolment validation depends on the Student 1 API.
- AI Mode depends on the local Ollama service.
- MCP Mode depends on the shared local MCP server.
- RAG Mode depends on the shared RAG server and Ollama service.

Assessment & Grades Management (Student 5):

- Grade creation and update depend on the Student 1 Student Records service being available for student ID validation.

- Course information enrichment depends on the Student 4 Course Management service.
- AI Mode depends on the local Ollama service being installed and running.
- If an external service is unavailable, some cross-feature information may not be displayed even though the core Assessment & Grades service remains available.
- MCP Mode depends on the shared MCP server being available when the integration is enabled.
- RAG Mode depends on the shared RAG server and local Ollama service being available when the integration is enabled.

14.2 Technical Debt and Limitations

Student Records Management (Student 1):

- There is limited support for audit history or traceability of record changes
- The AI mode is designed for demonstration, so it doesn't yet include proper validation or fallback behaviour for unexpected inputs or unrecognisable prompts

Staff Management (Student 2):

- AI requests may become less efficient as the staff registry grows. The local /ask route includes the full staff dataset in the prompt for each question. This is workable for a small demonstration dataset, but can increase prompt size and response time as records grow.
- Input validation could be more specific. Staff create/update operations check for required non-empty fields and duplicate email addresses, but do not appear to validate email format or constrain department and employment-type values to an approved set.

Timetable & Class Scheduling (Student 3):

- Timetable RAG embeddings are held in memory and rebuilt on every RAG server restart
- RAG similarity thresholds (0.55 / 0.70) were tuned for the local nomic-embed-text model and may need re-tuning for a different embedding model
- The shared MCP server, RAG server and agentic loop depend on each developer's local Python and Ollama setup

Course and Enrollment Management (Student 4):

- SQLite may not be suitable for large production workloads.
- MCP and RAG are designed for the local project environment rather than production use.

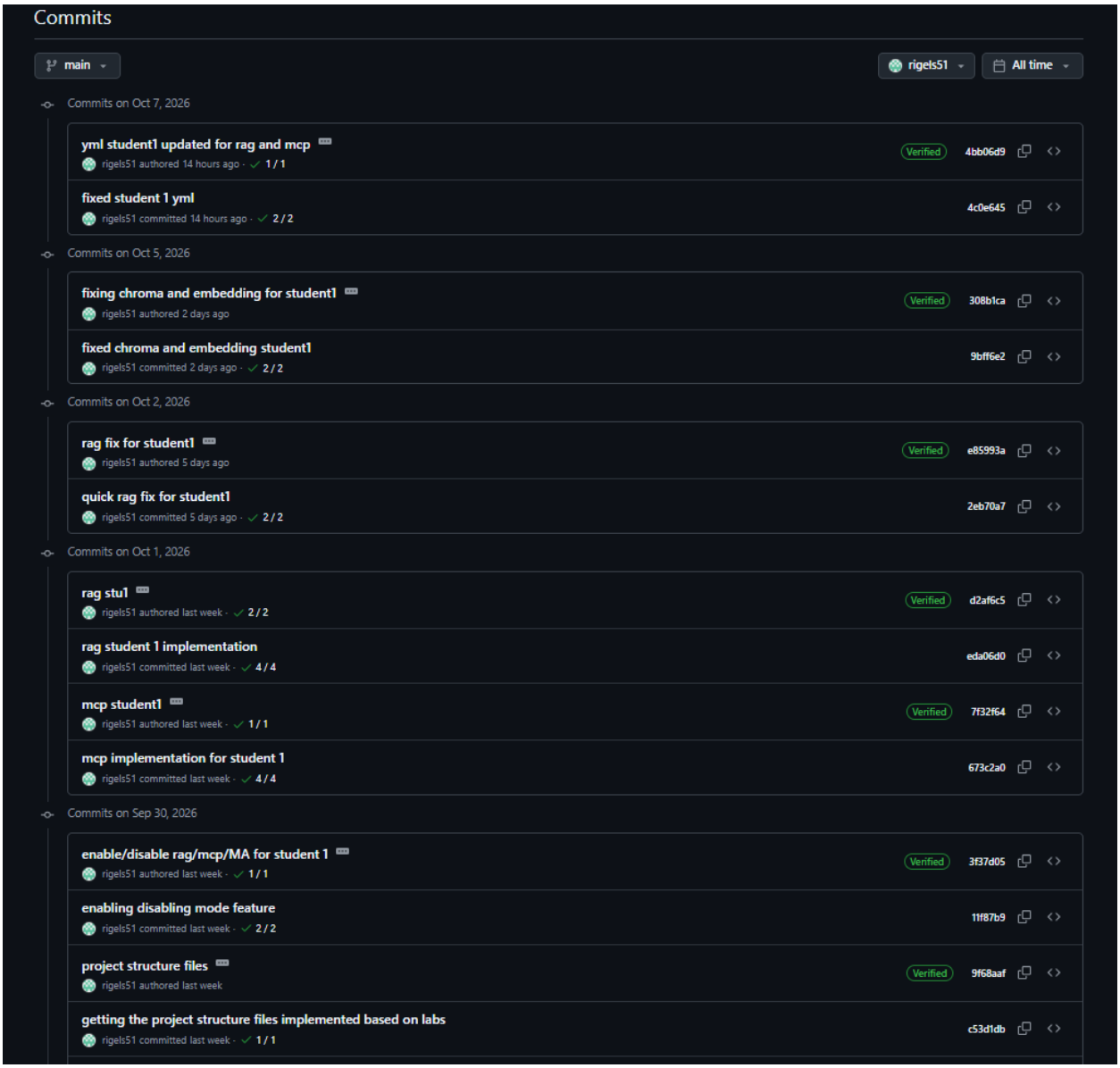
- RAG answer quality depends on the relevance of the retrieved context.
- Large-scale MCP and RAG performance has not been tested.

Assessment & Grades Management (Student 5):

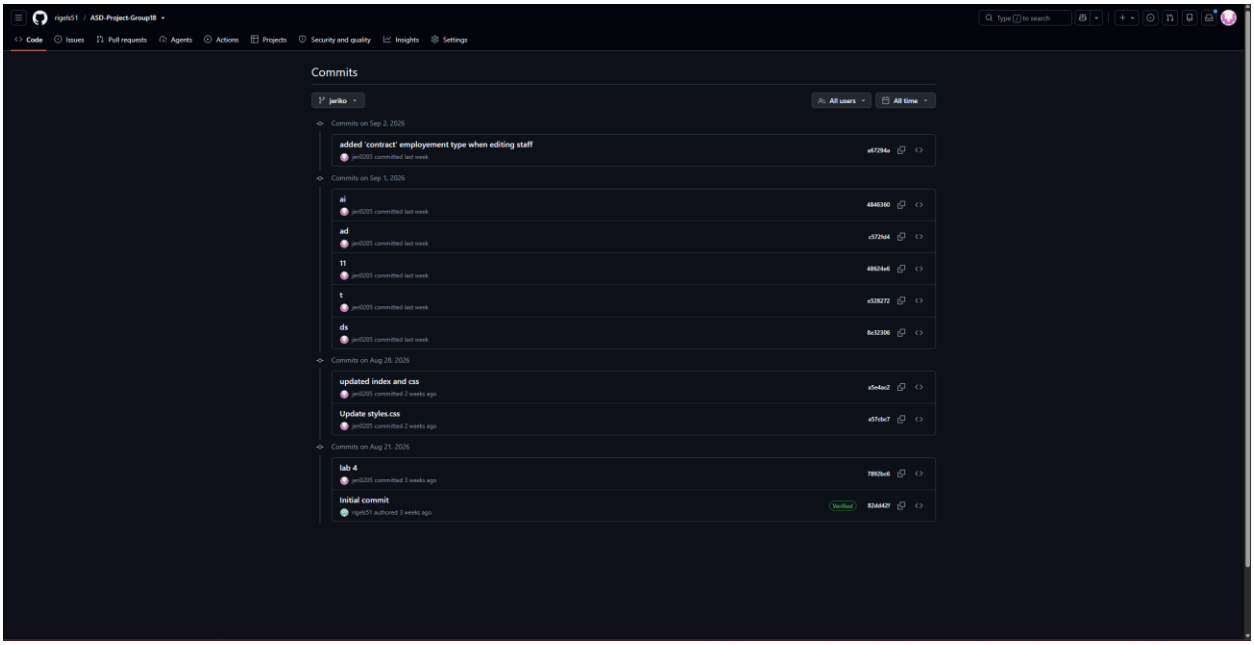
- SQLite is suitable for Release 0 but may not be appropriate for a large production university system.
- Student and course information is retrieved through direct REST API calls without caching.
- AI Mode is read-only and is designed to answer questions using retrieved assessment and grade information rather than perform write operations.
- For Release 1, MCP and RAG also depend on the shared project-level MCP and RAG services. The current implementation is intended for the project demonstration environment rather than a production-scale deployment.
- The RAG knowledge base intentionally excludes individual student marks, so individual student grade questions are not answered through RAG. Individual grade information is instead accessed through the appropriate controlled MCP tool.
- The RAG implementation has been tested using the available project dataset, but large-scale retrieval performance has not been evaluated with production-sized assessment and grade data.

15.0 GitHub Commit Logs (i)

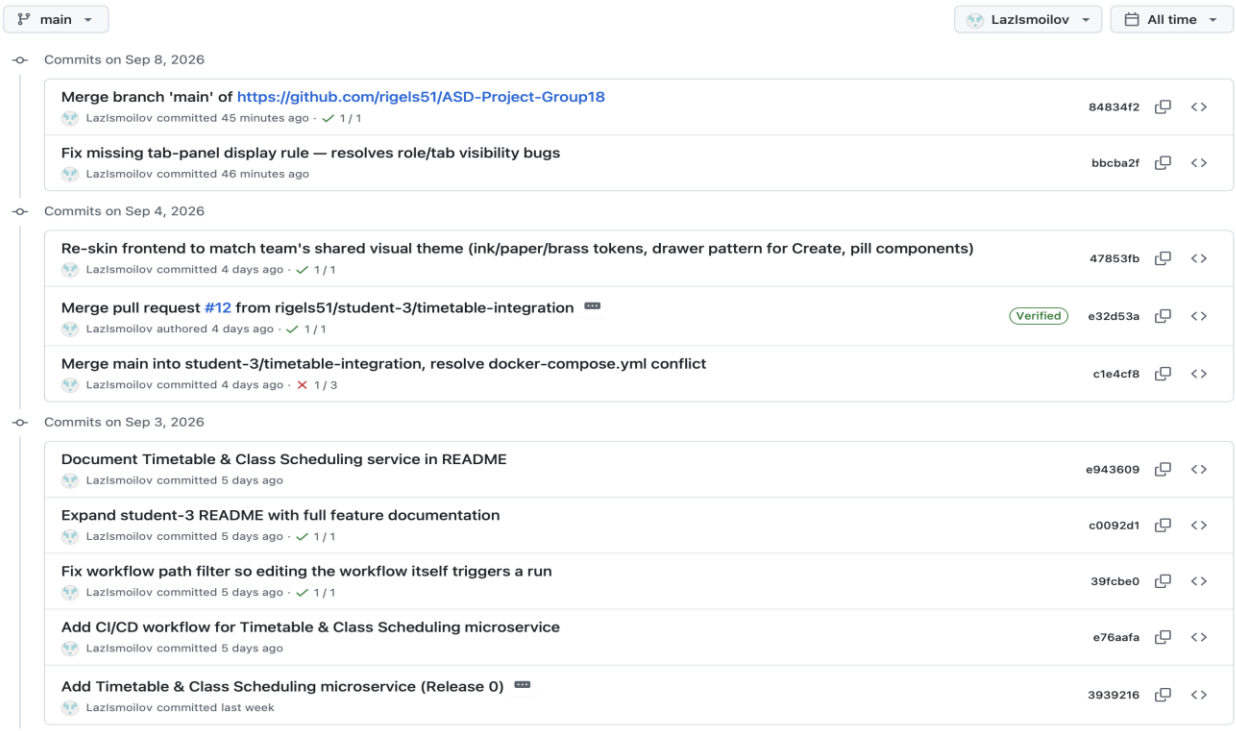
Rigel Rivamonte



Jeriko Arceo



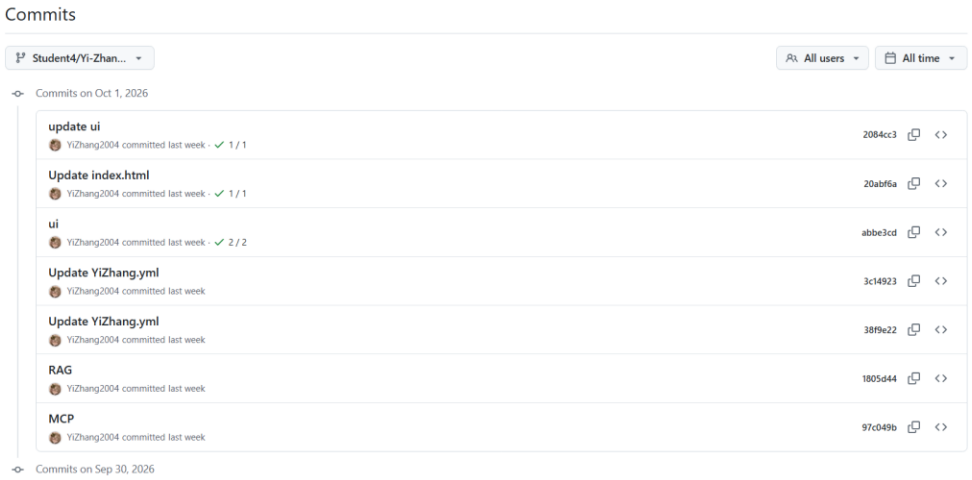
Lazizbek Ismoilov



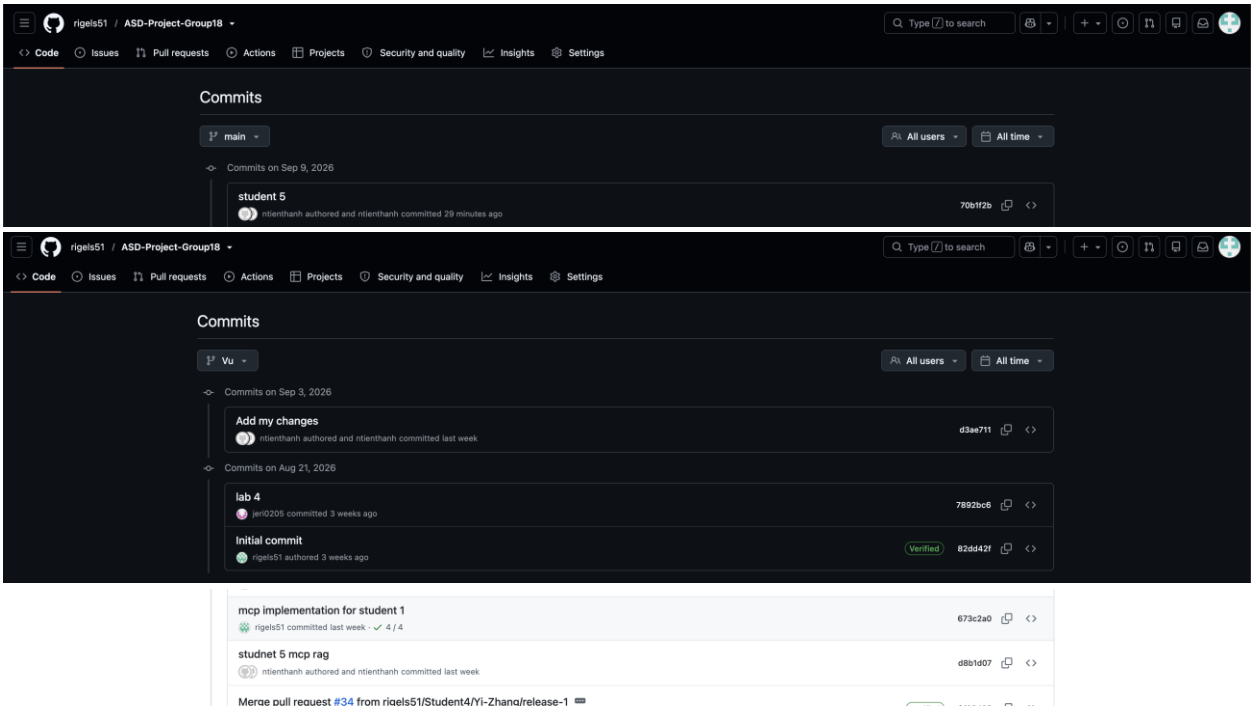
```
49ad19d  2026-10-01  Merge pull request #30 from rigels51/student-3/release-1
6b531c5  2026-10-01  Student 3: Release 1 agentic loop MCP and RAG validation outputs
9626ea9  2026-10-01  Student 3: reset Timetable result panels when the role changes
feecfef  2026-09-30  Student 3: Release 1 – Timetable on shared MCP/RAG + agentic loop MCP/RAG modes
84834f2  2026-09-08  Merge branch 'main' of https://github.com/rigels51/ASD-Project-Group18
bbcba2f  2026-09-08  Fix missing tab-panel display rule — resolves role/tab visibility bugs
```

Yi Zhang

The GitHub commit history below provides evidence of my Release 1 development work, including MCP and RAG integration, GitHub Actions updates, and frontend updates.



Vu Tien Thanh Nguyen



16.0 Contributions Logs (i)

Jeriko Arceo

My main contribution to the project was the Staff management Feature.

Contributions for the Application:

- I implemented the MCP tool exposure and RAG answer flow into my feature

Contributions for the Report:

- Created the structure for the document
- Completed my individual feature allocation
- Updated the functional and non-functional requirements for my feature
- Updated the risk management plan for my feature with
- Updated the individual software architecture diagrams for my feature
- Completed the MCP server and RAG server sections
- Completed the Grounded AI Response Flow
- Completed the GitHub Actions Workflow for my feature
- Created the showcase video for release 1

Lazizbek Ismoilov

My main contribution was the Timetable & Class Scheduling feature, built and integrated as an independent microservice from the ground up.

Release 1 contributions:

- Built three read-only Timetable tools on the shared MCP server and the “timetable” knowledge domain and policy corpus on the shared RAG server
- Added backend /api/timetable, /mcp/tools, /mcp/call and /rag/answer routes with MCP_ENABLED/RAG_ENABLED flags, and the MCP Tools and Policy Q&A frontend tabs
- Built the shared agentic loop's MCP and RAG validation modes, and the RAG server's domain routing that lets every feature plug in its own knowledge source
- Defined the team contract for the shared servers (MCP on 8000, RAG on 5050, a domain field on RAG requests, one tools file per feature), which all five features followed
- Updated student-3.yml so MCP and RAG are retained but disabled in CI; extended validate.sh to 27 checks (27/27 passed)

- Fixed a missing requests dependency that crash-looped the backend container (diagnosed with docker logs) and a role-switch bug that showed one role's results to another
- Recorded the Timetable segment of the Release 1 showcase video

Yi Zhang

My main contribution was the Course and Enrolment Management feature.

I developed the course and enrolment database and implemented the main CRUD functions.

I created the front-end pages for Course Management and Student Enrolment.

I also added AI Mode using Ollama to answer questions using current course and enrolment data.

For Release 1, I added MCP Mode and RAG Mode and connected the feature to the shared MCP and RAG servers. I also implemented MCP tools and RAG retrieval for course and enrolment information.

I separated the feature into frontend, backend, and database services and updated the Docker configuration and GitHub Actions workflow for Release 1.

Rigel Rivamonte

My main contribution was the Student Records Management feature.

I was mainly responsible for the Student Records feature end-to-end: its database, backend, and frontend microservices, and the implementation of the MCP Mode and RAG Mode integrations with the shared MCP and RAG servers, including the ChromaDB-backed retrieval pipeline. I also set up and maintained Student 1's CI pipeline (RigelRivamonte.yml), extending it in Release 1 to validate and build the new MCP/RAG components.

Vu Tien Thanh Nguyen

My main contribution was the **Assessment & Grades Management** feature.

Contributions for the Application:

- I designed and implemented the Assessment and Grade data structure.

- I developed the Flask database service using SQLite for storing assessment and grade information.
- I implemented CRUD operations for assessments and grades.
- I developed the Assessment & Grades frontend interface.
- I integrated the feature with Student 1's Student Records service to validate student IDs and retrieve student information.
- I integrated the feature with Student 4's Course Management service to retrieve course information.
- I implemented AI Mode using Ollama and the qwen2.5:0.5b model.
- I implemented the Plan → Act → Observe → Adapt workflow so AI questions can use current assessment and grade information.
- I containerised the frontend, backend, and database services using Docker.
- I integrated the Student 5 services into the group's shared Docker Compose application.
- I created automated tests for the Assessment & Grades API.
- I created the GitHub Actions CI workflow for Student 5.
- Integrated the Assessment Service with the shared MCP and RAG servers.
- Implemented three read-only MCP tools for assessments, grade summaries, and upcoming assessments.
- Added MCP input validation and tool restrictions.
- Implemented the Student 5 RAG domain using assessment records, anonymised grade statistics, and assessment policy information.
- Added citations, sources, confidence levels, and insufficient-evidence handling for RAG responses.
- Updated Docker Compose, automated tests, CI, and Agentic Loop review cases for Release 1.

Contributions for the Report:

- Completed my individual feature allocation.
- Completed the functional and non-functional requirements for my feature.
- Completed the feature plan for Assessment & Grades Management.
- Completed the risk management plan for my feature.
- Completed the individual software architecture description and diagram for Student 5.
- Documented the Student 5 GitHub Actions workflow.
- Added Student 5 implementation evidence, GitHub commit evidence, known issues, and limitations.

- Added Release 1 MCP and RAG requirements and updated the feature plan.
- Documented the MCP tools, RAG workflow, architecture, and Docker Compose configuration.
- Added Release 1 testing, CI, Agentic Loop, implementation, and GitHub evidence.
- Updated the Student 5 risks, known issues, limitations, and contribution records.

17.0 Showcase Video

YouTube link - https://www.youtube.com/watch?v=kQF2dLFFDik

